A Schematic Language for Consistent Hardware Design

Traditional hardware description languages (HDLs) continue to dominate industrial practice, although modern alternatives have demonstrated clear value in many scenarios. To understand this persistence, we conduct the first empirical study on the HDL pragmatics of professional hardware development, interviewing 30 developers from 20 companies and institutes. The study identifies consistency as an important requirement that existing HDLs have not pursued as a primary design goal alongside other important qualities. That is, source code should reflect design intent and correspond predictably to circuit schematics, allowing developers to anticipate circuit structure when making design decisions and trace schematic-level issues back to the code for debugging or optimization. In mainstream practice, this consistency is maintained largely through institutional experience and coding guidelines rather than a language-level account.

Thus, we propose NetX, an HDL that adopts a schematic programming model and treats consistency as a first-class design principle. It features a core language resembling circuit schematics and curated meta-programming mechanisms for effective abstraction without sacrificing consistency. We formalize its core and prove several consistency-related properties; further, we establish schematic properties such as acyclicity and planarity through source-level reasoning. We mechanize the results in Rocq (15K LoC) and implement a prototype toolchain (30K LoC), including a compiler, a package manager, a standard library, a simulator, a visualizer, and language server support. We evaluate NetX on hardware designs from several application domains to show its feasibility for actual hardware development, and conduct a user study that provides preliminary evidence that users perceive the consistency benefits of NetX.

1 Introduction

Verilog [39] and VHDL [40], the traditional hardware description languages (HDLs) that originated in the 1980s, continue to dominate industrial practice, even as the hardware landscape has become more complex and diverse [10, 11, 26, 46] and modern alternatives such as Bluespec [63], Chisel [3] and Clash [2] have demonstrated clear value in many projects and textbooks [7, 16, 28, 49, 70, 85, 89]. The persistence of traditional HDLs alongside continued language innovation makes it worthwhile to continue examining the qualities that HDL design should support. In particular, alongside goals such as modularity [34, 61], concurrency management [7, 74], and agile development [83, 84], what other qualities do hardware engineers value but existing HDLs have not explicitly pursued as primary design goals? In this paper, we examine one such quality and investigate how principled language design can better support it.

To identify such a quality, we conduct the first empirical study on the pragmatics of professional hardware development, recruiting 30 developers from 20 companies and institutes, such as AMD, Huawei, ICT, Nvidia, Synopsys, and ZTE. The study identifies *consistency* as one of the most emphasized concerns: that is, intent, code, and the resulting circuit should stay consistent, such that developers can predict the generated circuit early to make better design decisions and trace the circuit back to the code to localize and fix issues more effectively. This requirement is distinct from conventional concerns such as functional correctness [14, 27] and synthesizability [19, 77]: the former concerns whether the outputs are correct, and the latter whether the source can be translated into hardware, with neither requiring the internal circuit schematic to remain explicit to developers.

Our empirical findings further indicate that this consistency is currently maintained largely through coding guidelines. In traditional HDLs, guidelines help developers associate particular coding patterns with expected circuit schematics; this is useful but highly institution-specific, relying on conventions of particular teams and tools rather than a general basis for understanding and reasoning. Newer HDLs, on the other hand, often provide more powerful abstractions to improve productivity, but they lack comparable guidelines for maintaining consistency; moreover,

their abstractions can obscure the exact schematics. Meanwhile, lower-level representations are consistent with circuits, but they are too verbose for manual development or source-level reasoning.

The inconsistency stems from a mismatch in programming models as shown in Fig. 1: circuits are essentially schematic, while mainstream HDLs are largely behavioral due to historical reasons [23]. Design intent is commonly documented in schematic descriptions such as architectural diagrams, and compiled hardware is ultimately represented as a netlist, i.e., a graph of interconnected components. Yet the source language that mediates between them often describes behaviors rather than schematics. For example, the behavioral construct `if-else` can describe a combinational circuit by specifying its truth table, but the table can correspond to substantially different circuits, each with distinct performance and resource implications. Since the exact circuit is only determined after time-consuming black-box hardware synthesis, this mismatch makes it more difficult for developers to predict the generated circuit and to trace backend diagnoses to the source code, thus becoming the root cause of the inconsistency problem.

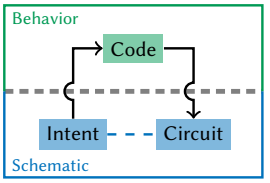


Fig. 1. The insight into the root cause of inconsistency.

Motivated by this observation, we propose NetX, a hardware description language that adopts a schematic programming model and treats consistency as a first-class design principle.

NetX is not intended to replace existing HDLs, but rather to complement them. When consistency is the primary concern, NetX can be the primary choice; when only critical components need extra consistency, NetX can be integrated with other HDLs; and when other goals are prioritized, NetX can still serve as a consistent IR to yield predictable compiled results. NetX code is organized in a grid schematic (breadboard) where components are aligned vertically and horizontally, while names are jumper wires for more complex topologies. Specifically, NetX's design consists of two key aspects:

- *A Schematic Core Language.* NetX introduces a core language resembling circuit schematics and gives it a syntax-directed translation to circuit graphs, intuitively providing consistency. It describes the assembly of circuit components, empowering developers to manipulate exact circuit schematics without obscuration. Formally, we define a core language with straightforward elaboration and graph-construction semantics; we prove various consistency-related properties and show how it allows source-level reasoning about schematic properties such as acyclicity [15] and planarity [69]. All these results are mechanized in Rocq [79] in 15,000 lines.
- *Curated Abstraction Mechanisms.* To enhance usability, NetX also adopts appropriate abstractions for schematic patterns. Since NetX code resembles schematics, schematic patterns are subsequently reflected in code patterns, making meta-programming a natural fit for abstraction. Curated meta-programming mechanisms operate syntactically and can be desugared by straightforward elaboration, providing effective reuse and modularity without sacrificing the explicit schematic correspondence provided by the core language.

Further, we implemented a prototype toolchain for NetX in approximately 30,000 lines of code, including (1) a compiler that parses, elaborates, and analyzes NetX code through multiple passes, generating backend representations; (2) a simulator that interprets the generated circuit and checks functional correctness before deployment; and (3) a package manager, a standard library, a visualizer, and language server support for convenience and usability. To demonstrate the feasibility of NetX for actual hardware development, we exercised this toolchain on a range of hardware designs spanning signal processing, video compression, communication protocols, cryptography, and computer architecture, up to a case study of an FPGA-deployed computer system built around a non-trivial pipelined CPU with various IO devices. We also conduct a user study that provides preliminary evidence that users perceive the consistency benefits of NetX.

In summary, our work makes the following contributions:

- We conduct the first empirical study on HDL pragmatics in professional hardware development. The study not only inspires NetX, but also may provide insights for future work in the community.
- We propose NetX, an HDL that adopts a schematic programming model and treats consistency as the design principle. It features a core language intuitively resembling circuit schematics and curated meta-programming mechanisms for effective abstraction without sacrificing consistency.
- We formalize the core of NetX and prove its consistency-related properties, and show how to reason about schematic properties from source code, with all results mechanized in Rocq.
- We implement NetX as a prototype and evaluate it through a range of hardware designs, a case study, and a preliminary user study. The results provide evidence that the prototype toolchain can support the development, testing, and deployment of hardware designs, while preliminary user feedback suggests that participants perceived NetX's intended consistency benefits.

The nine-page supplementary material includes two additional pages of semantic details, six additional supporting theorems, all proofs, and further details of our case study and user study. We will fully open-source NetX and submit the complete artifact (see Data-Availability Statement).

2 Empiricism

We conduct an empirical study to understand why the hardware industry continues to use traditional HDLs, despite the emergence of various modern HDLs. This is the first empirical study on the pragmatics of professional hardware development; previous studies in academia primarily focus on novices and amateurs [6, 54, 59], while no relevant industry report is publicly available as far as we know. Due to space constraints, we only present empirical findings related to the scope of this paper, with the complete record available in the artifact.

2.1 Research Questions

We examine three factors influencing adoption of new HDLs, illustrated in Fig. 2, through the following research questions (RQs):

- RQ1 (Push of Developers): To what extent do developers find the pitfalls in traditional HDLs a problem?
- RQ2 (Drag of Old HDLs): To what extent does the ecosystem of traditional HDLs obstruct the transition?
- RQ3 (Pull of New HDLs): To what extent do the advantages and modern features of new HDLs encourage industry adoption of these alternatives?

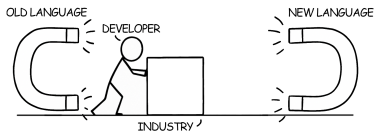


Fig. 2. A comic showing the factors affecting the industry's adoption of new HDLs, drawn with the aid of ChatGPT.

2.2 Methodology

Participant Recruitment. To mitigate potential bias, participants were recruited from diverse backgrounds, primarily focusing on industry professionals with additional representatives from academia to provide a broader perspective. As summarized in Table 1, a total of 30 participants from 20 different institutions and companies unrelated to the authors, such as AMD, Huawei, ICT, Nvidia, SEU, Synopsys, and ZTE, took part in the study. All of them are frontline hardware developers, with experience ranging from 0.5 to 6 years and averaging 3.2 years (by 2025). Although we reached out to some more senior engineers, they declined the interview since they had moved into more managerial roles. All participants have consented to the recording and use of their input for research purposes.

Interviews and Questionnaires. We conduct open-ended, semi-structured interviews using predefined questions with follow-ups adapted to participants' responses. This approach allows for exploration of unexpected insights while reducing potential bias [87]. The interviews took 19.7 minutes per participant on average (standard deviation: 3.7 minutes), excluding introduction and preparation.

Table 1. Overview of participants' information, showing their years of experience in hardware design (Exp), the HDLs they use in work (Language), and their duties (Background).

Index	Exp	Language	Background	Index	Exp	Language	Background
P01	2	Verilog [39], SV [41]	Researcher, CPU	P16	6	Verilog	Engineer, IC Frontend
P02	3	Chisel [3], Verilog	Researcher, CPU	P17	3	Verilog	Engineer, IC Frontend
P03	5	Verilog	Researcher, CPU	P18	3	Verilog	Engineer, IC Frontend
P04	3	Verilog	Researcher, CPU	P19	3	SV	Engineer, IC Frontend
P05	4	Verilog, SV, Bluespec [63]	Researcher, XPU	P20	4	Verilog	Engineer, IC Frontend
P06	2	Verilog	Researcher, FPGA	P21	3	Verilog	Engineer, IC Frontend
P07	4	Verilog	Researcher, FPGA	P22	3.5	Verilog	Engineer, IC Frontend
P08	1.5	HLS [18], Verilog	Researcher, FPGA	P23	2.5	Verilog	Engineer, IC Verification
P09	2	Chisel, Verilog	Researcher, FPGA	P24	3.5	SV	Engineer, IC Verification
P10	2	Verilog	Researcher, IC Design	P25	2.5	SV	Engineer, IC Verification
P11	0.5	Chisel	Researcher, IC Verification	P26	6	Verilog, SV	Engineer, IC Verification
P12	1.5	Verilog	Intern, IC Frontend	P27	4	Verilog, SV, Chisel	Engineer, IC Design
P13	2	Verilog	Intern, FPGA Education	P28	3	Verilog	Engineer, IC Design
P14	4	Verilog	Engineer, FPGA	P29	5	Verilog	Engineer, IC Backend
P15	5	Verilog, SV, HLS	Engineer, FPGA	P30	2	Verilog	Engineer, IC Backend

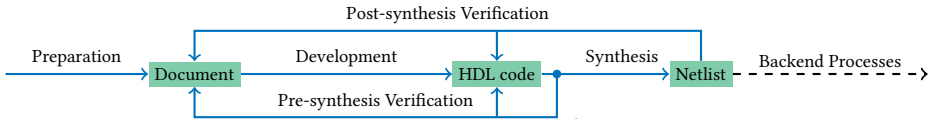


Fig. 3. Overview of a typical hardware design workflow, where blue solid lines indicate HDL-related processes.

Questions were tailored to participants' roles in the workflow in Fig. 3: HDL coding for frontend engineers, testing and debugging for verification engineers, and the effects of frontend decisions on physical design for backend engineers. For researchers and full-stack engineers, we combined these topics.

To guide the interviews, we also set 5-minute questionnaires beforehand to collect participants' background information and knowledge. Besides, the questionnaires also serve as a quantitative supplement to RQ1; we will later discuss the results in Section 2.3.2.

Threats to Validity. The study may be subject to several threats: (1) The sample size is limited. (2) Some participants may not be fully aware of the most recent advances in HDLs, leading to potentially outdated opinions. (3) Due to the flexible nature of semi-structured interviews, the interviewer's on-the-spot decisions in follow-up questions and interpretation may not be identical.

2.3 Findings

2.3.1 Overview. In summary, we found that (1) participants are often unaware of design pitfalls in traditional HDLs; (2) decades-long industry experience with these languages can somehow prevent common pitfalls; (3) new HDLs typically focus on expressiveness, misaligning with some key requirements of the industry. Further reviewing the interview records, we found that a property often revisited or implicitly reflected by many participants indicates one of the key requirements of the industry, i.e., *consistency*, the property we focus on in this paper.

Consistency emphasizes the correspondence between the documented intent, the code, and the generated circuits. 30/30 participants mentioned consistency between documents and code, and 25/30 further mentioned consistency between code and circuits. They expect documents to be implemented exactly by code and synthesized to intended netlists; also, circuits should be traceable back to the code. This helps to understand, test, and debug hardware designs, reducing potential economic losses in later stages. For example, in Fig. 3, pre- and post-synthesis verification, which essentially check consistency and trace bugs across the three representations, account for most of the time and labor throughout the workflow. Consistency is largely maintained through developers' experience and coding guidelines for traditional HDLs. For newer HDLs, comparable coding guidelines and principled language support for consistency are still lacking; explicitly incorporating such support into the language design could make new HDLs more attractive to some industry developers. The insight further shows an opportunity for consistent HDL design.

2.3.2 RQ1: Push of Developers. Generally, participants did not find the coding experience with traditional HDLs to be an obvious problem in their workflow, since: (1) they were only partly aware of pitfalls in traditional HDLs; and (2) the limited expressiveness of traditional HDLs was not a major concern that would drive them toward newer languages.

Unawareness of Pitfalls. To examine developers' awareness of pitfalls in traditional HDLs, we first use the questionnaire to check their knowledge of specific pitfalls in Verilog (the language known to all participants), and then interview them to further understand their awareness.

Participants' responses to the questions are shown in Table 2, where: Q1 comes from a textbook example [80], a question intended to filter out unqualified participants—however, it turns out nearly half of the participants, including some senior engineers, cannot answer it correctly; Q2 and Q3 examine participants' knowledge of the two representative pitfalls in Verilog selected from previous research [13, 56], concerning deeper language design issues; fewer participants answered correctly.

Table 2. Questionnaire results, where #valid is the number of definite answers, and #correct and %correct are the number and percentage of correct answers, respectively.

	Q1	Q2	Q3
#valid	30	30	29
#correct	16	8	11
%correct	53.3	26.7	37.9

We further interviewed the participants to understand why they failed to answer these questions. Taking Q1 as an example: in `always @(*) a[i]=b+c;`, whenever the sensitive signals (`b`, `c`, and `i`) change, the assignment is triggered. Since this structure is so common in Verilog, it is quite surprising that only 53.3% of the participants noticed that `i` is a sensitive signal in the code. Those who overlooked this detail, including some senior engineers, explained, "We have a coding guideline that only allows constant left-hand-side indices, so I have never noticed this" (P24). As for Q2 and Q3 concerning deeper language design issues, participants mentioned similar guidelines (P03, P15), i.e., experience-based rules restricting language usage, which we will discuss later in Section 2.3.3.

Deprioritization of Expressiveness. Though the design of traditional HDLs is less expressive compared to modern software languages, participants did not find it a major concern.

A primary reason is that HDL coding is a relatively straightforward task under the guidance of a comprehensive design document. Hardware documentation is generally more detailed than software documentation, including not only overall specifications (e.g., architectural diagrams, timing diagrams, and pipeline structures) but also in-depth descriptions of critical components (e.g., critical paths, state machine models, and protocols). While academic settings may feature less detailed documents, essential diagrams and specifications remain indispensable—all participants except P11 reported having such documents before coding. One participant noted, "HDL coding is basically translating the design document" (P20). Another participant remarked, "During documentation, we discuss critical design choices, including security concerns and protocols, which is more important than coding itself" (P28). One even stated, "While beginners can manage to write Verilog code, effective documentation is a task that only senior engineers can handle well" (P03).

Given this context, it is unsurprising that participants distrust expressive language features. They remarked, "Features like recursive functions (which are even synthesizable) in Verilog are bad practices; they lead to unexpected circuits with unexpected problems" (P04). "RTL coding only takes a minor part in the overall workflow; using high-level features may save coding time, but it is not worth the extra verification (debugging) efforts" (P17, P18). Notably, some senior verification engineers reported the biggest challenge in their work is that "different engineers may have different understandings of the document" (P20, P22), which is not even related to HDL code.

2.3.3 RQ2: Drag of Old HDLs. Through decades of practice, experience-based approaches have been built around traditional HDLs to address certain design challenges. The experience takes time to build and is tightly coupled to traditional HDLs, preventing the industry from abandoning them.

Developers need to translate the document into HDL code in a way that synthesis tools can generate intended netlists, which basically relies on developers' experience with traditional HDLs. For instance, they often need to predict the logic depth (the number of logic gates in the longest path of the circuit) to meet timing constraints. In some cases with extreme PPA (power, performance, and area) constraints, developers even need to synthesize every line of code on the fly in their mind to ensure the circuit is as intended (P01, P10). However, as participants stated, "Only a few very senior developers who are familiar with the backend tools can accurately predict and write the exact code needed" (P29), even given that "To predict the generated circuit is the most efficient way to optimize the design" (P02); and that "Failing to do so and modifying the design in later stages can be extremely difficult and costly" (P18).

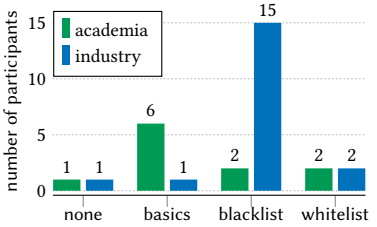


Fig. 4. Strictness of reported guidelines.

Often, the experience of senior engineers are codified into coding guidelines. The guidelines identify error-prone coding styles and codify certain correspondences between specific code patterns and generated circuits. All participants (even some using new HDLs) except for P11 and P14 reported having such guidelines. Fig. 4 shows the overview of participants' coding guidelines. (1) The guidelines can be basic, which are typical in academia—these guidelines contain textbook rules such as to write an `else` branch for every `if` statement in the combinational circuit to avoid latches. (2) In industry, the guidelines are stricter, often in the form of a blacklist forbidding certain coding styles—for example, it is not allowed to have implicit variable declarations or type conversions; and it is not allowed to specify a finite state machine in a one-process style. (3) Moreover, in some cases requiring very high reliability, whitelist guidelines only allow a very limited subset of the language.

Participants emphasized the importance of coding guidelines, stating that "Though I did not notice some language flaws, the strict coding guidelines prevent me from making relevant mistakes" (P15, P20, see Section 2.3.2). For instance, P17 once encountered strict timing constraints and had to replace all `if-else` statements with `?:` to guide the synthesis; while P10, who works with such constraints every day, has a guideline that only allows `?:` for combinational circuits, avoiding that problem. Moreover, an intern mentioned even in common development that "Since I am not familiar with the coding guideline, the generated circuit diverged far from my expectation" (P12), which is also confirmed by a senior engineer: "Interns are almost unavoidably prone to making mistakes since they lack experience with the correspondence between code and circuit" (P16). A participant who transitioned from industry to academia observed that the guidelines become less strict, which can "lead to more bugs and readability issues in the code, requiring more effort in verification and debugging. For example, it becomes more difficult to trace a circuit issue back to the code" (P27).

The results indicate that participants generally have to rely on experience and guidelines to achieve acceptable consistency; otherwise, they may encounter various pitfalls. This, in another aspect, reflects the lack of language guarantees for consistency in traditional HDLs (and in new HDLs, see Section 2.3.4), showing the opportunity for consistent HDL design.

2.3.4 RQ3: Pull of New HDLs. New HDLs appear not to be very attractive to participants (especially those from industry) due to concerns about the increased inconsistency and debugging efforts.

Extra Inconsistency. New HDLs are typically built on top of traditional HDLs, introducing high-level software-like abstractions for enhanced expressiveness, which can introduce even more inconsistency due to the extra abstraction. As a participant involved in hardware design education remarked, "The new HDLs provide a more software-like programming experience and can ease the design of some control signals. Actually, our primary goal is to break the software mindset of the students and to think in a hardware way, to consider the structure, the concurrency, etc." (P13).

Participants remarked that inconsistency can be a major factor when it comes to industry products, stating that "Due to the higher-level abstraction, we can hardly control the cost and redundancy of the generated circuit, making it difficult for the industry to accept them" (P14); and that "Fabrications, foundries, and standard cells require clear specifications to ensure certain code patterns make specific circuits, which is difficult to codify for a new HDL" (P29). Another piece of evidence is that our questionnaire also included a Chisel question adapted from a textbook example [2], examining whether participants noticed that the `switch` construct in Chisel may generate redundant logic—among the 16 participants who know Chisel, only 6 noticed that.

Given these perspectives, it is understandable that participants who write Chisel almost never use its advanced features, stating, "Writing Chisel is basically the same as writing Verilog. I have written Chisel for years, but only recently I started trying some template features, very rarely" (P09); and that "Our company starts to use Chisel in some non-critical projects, yet Chisel code is almost the same as that in Verilog—the greatest advantage of Chisel is the IDE support (inherited from Scala), which effectively reduces typos. It is like Verilog with an on-the-fly linter" (P21).

Extra Debugging Efforts. This extra inconsistency issue subsequently leads to heavier debugging efforts. Testing and debugging are called verification in hardware design, which is already heavy enough for traditional HDLs—it is typically the most time-consuming part of hardware design [24, 25]. One of the major challenges in verification is debugging an issue identified in the netlist. For example, one participant shared, "Once, I found a signal was directly used by thousands of components, which was fine in simulation (akin to passing a variable to many functions), but was problematic in the netlist due to fan-out constraints" (P14). In such cases, "the first choice is to directly modify the netlist, since modifying the HDL code may not actually fix the problem and can introduce new bugs (due to the lack of code-to-circuit correspondence); plus the need to re-verify and re-synthesize, it becomes very time-consuming. However, complex problems can still require source-level changes to guide synthesis toward the intended circuit." (P17).

To make things worse, new HDLs may bring even more challenges. Participants noted, "Coding is only a small part of the workflow—new HDLs may save some development time but introduce more debugging issues, ultimately costing more time overall" (P17, P18). For example, Chisel code is first compiled into behavioral Verilog and then synthesized into a netlist. This brings further inconsistency, as stated, "When coding in Verilog, different developers often write similarly, but people write Chisel very differently even when implementing the same circuit, which makes verification and debugging harder" (P27). Another participant shared, "Once I found the Chisel compiler generated an intermediate variable in the Verilog code with an unexpected width, it was really tricky to identify that problem" (P11). Yet another participant echoed, "Sometimes we need to debug the Verilog code generated by the Chisel compiler, but that is almost unreadable" (P09).

2.3.5 Implications for Design. The findings motivate consistency as a first-class language design principle: source code should reflect documented design intent, expose predictable circuit structure, and support tracing circuit-level issues back to the source. Other suggested aspects of language design, such as toolchain integration, IP reuse, and state-machine modeling, are also inspiring for the design and implementation of NetX and its toolchain (see Section 5 and the supplementary).

3 Language Design of NetX

In this section, we introduce NetX, a novel HDL that adopts a schematic programming model. We first discuss the motivation, explaining why this model is suitable for scenarios emphasizing schematic consistency (Section 3.1). Then, we present the core model (Section 3.2.1) and the appropriate abstractions enhancing usability without sacrificing consistency (Section 3.2.2).

3.1 Motivation

NetX seeks to resolve the root cause of the current inconsistency issue—the misalignment between the behavioral programming model and the schematic nature of hardware (shown in Fig. 1). While hardware is schematic, the programming model of existing HDLs is not.

Hardware is schematic. According to our empirical findings in Section 2, design intentions are typically documented in schematic drafts; meanwhile, the backend circuit representation is typically a network of interconnected components, which is inherently schematic. However, HDL code typically follows a behavioral model that specifies circuits by describing their behavior, which obscures the actual circuit schematics. For instance, as illustrated in Fig. 5, both an encoder (left) and a decoder (right) can be described using similar behavioral code. However, their underlying schematics are vastly different: the decoder is noticeably more complex than the encoder. This gap arises because `switch` can describe any circuit by specifying its truth table, but the table can correspond to various circuits, each with distinct properties.

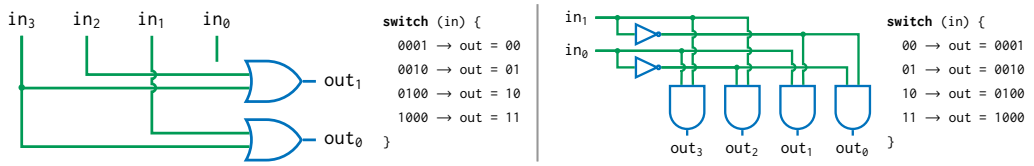


Fig. 5. Illustration of how behavioral code misaligns with circuit schematics.

A similar issue also arises in structural HDLs. For example, structural Verilog still does not avoid semantic flaws that harm consistency since it remains a subset of behavioral Verilog and thus inherits behavioral pitfalls such as process scheduling issues (e.g., the confusing zero delay `#0`) [13]. Additionally, it can be too low-level for manual development. Meanwhile, functional HDLs such as Ruby [27] can obscure consistency through abstraction. Ruby uses higher-order functions to specify wire crossing and forking (e.g., for the encoder, `enc = fork; [snd pair, fst fork]; zip; map or`), which makes the schematic structure explicit. However, its relational model also allows the decoder to be specified as the converse of the encoder, i.e., `dec = enc`$^{-1}$, which again hides the decoder's distinct underlying structure and thus weakens the correspondence between code and schematics.

Given this misalignment, there are two major consequences related to the inconsistency issue:

- *Weakened control over circuits.* The behavioral model obscures the exact circuit schematics, causing more development and debugging effort. As reported in Section 2.3.3, developers have to additionally rely on experience-based coding guidelines to maintain consistency; and as reported in Section 2.3.4, they often struggle to trace bugs from circuits back to code. Moreover, known language design flaws are highly related to the behavioral model, which not only troubles hardware developers but also confuses developers of language tools [13, 55].
- *Underutilized abstractions.* High-level programming mechanisms designed for behavioral abstraction may not be suitable for capturing schematic patterns in hardware design and can introduce additional inconsistency. Due to this concern, high-level programming features in HDLs are generally underutilized in practice—developers only adopt a very limited subset of language features, as reported in Section 2.3.2 and Section 2.3.4. While even basic constructs (e.g., control-flow constructs like `switch` and `break` [41]) already pose inconsistency, other common high-level behavioral constructs, such as concurrent processes [39, 40], behavioral subtyping [41, 70], and relational algebra operators [2, 78], may further exacerbate the problem in various scenarios.

Existing work has partially addressed this inconsistency issue from different directions, yet schematic consistency is not treated as a first-class design principle in these languages. Prior efforts on compiler patches [43], static scheduling strategies [13], and tailored semantics [56] could

mitigate some inconsistency issues; however, these approaches do not eliminate the fundamental mismatch in the programming model, and the inconsistency issue still persists—leading to the persistent reliance on institutional experience and coding guidelines.

These existing languages remain valuable and effective in many scenarios and our goal is complementary: to explore a design point where schematic consistency is treated as a first-class concern. In this context, we argue that the inconsistency issue can be better addressed through a principled language design that closely aligns the programming model with the hardware schematic. Thus, we propose a novel language NetX that (1) adopts a schematic core, enabling fine-grained control over circuit schematics (Section 3.2.1); and (2) extends the core with curated meta-programming mechanisms, allowing expressive abstraction without sacrificing consistency (Section 3.2.2).

3.2 Schematic Programming Model

The schematic programming model of NetX is designed to closely resemble circuit schematics. With rigorous discussion deferred to Section 4, here we provide intuition on how the schematic model works, that is, how a NetX program is constructed and how to assemble complex systems from basic components while the code keeps resembling circuit schematics.

3.2.1 Core Language. First, we present the core constructs of the programming model of NetX. The basic constructs in NetX are expressions corresponding to circuit components, i.e., schematic blocks. Primitive expressions are identifiers of either basic components (e.g., AND, OR, and NOT) or user-defined components composed from these gates. Here, each identifier in NetX corresponds to a component in the schematic.

The composition of components is described by two fundamental combinators `<>` and `||`, representing the serial and parallel composition, respectively. Fig. 6 gives a step-by-step illustration of assembling a 4-input AND gate using three 2-input AND gates. In the figure, starting with a 2-input AND gate (left), we first use the parallel combinator `||` to compose two such gates, resulting in a 4-to-2 component (middle). Next, by connecting the two outputs to another 2-input AND gate using the sequential combinator `<>`, we obtain a 4-to-1 component as intended (right).

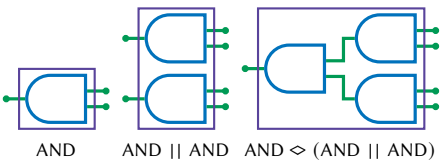


Fig. 6. Example of assembling a 4-input AND gate using NetX's schematic expressions.

```
1 component HA : [x1, x2] → [sum, carry] {
2   wire x1, x2, sum, carry of bit(1);
3     sum <> XOR <> (x1 || x2);
4   carry <> AND <> (x1 || x2);
5 }
```

```
let HA =
(XOR <> (x1 || x2))
      ||
(AND <> (x1 || x2))#
```

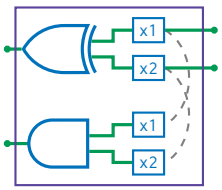


Fig. 7. The half adder. The **let** expression is the desugared version of the component definition, corresponding to the schematic on the right.

NetX uses component as the unit of modularity, defining reusable circuit components. Each component has a list of named inputs and outputs, as shown in the above half of Fig. 7. Here, Line 1 defines a component named HA, a half adder that takes two inputs x1 and x2, and produces their sum sum and carry-out carry. Line 2 declares x1, x2, sum, and carry as named wires, each carrying a 1-bit-wide signal. Notably, the named wires are a special kind of 1-to-1 component that forwards its input directly to its output, commonly used to describe circuits with complex interconnections. Finally, Lines 3 and 4 contain the two expressions assembling a standard half adder, producing the outputs.

Note that component definitions can be desugared into inline name bindings; later, when formalizing NetX, we also apply this desugaring for simplicity. For example, Fig. 7 shows the desugared expression of the HA component defined above, where **let** HA = ... binds the schematic of a half adder to the name HA. Here, a new operator # is introduced: the original

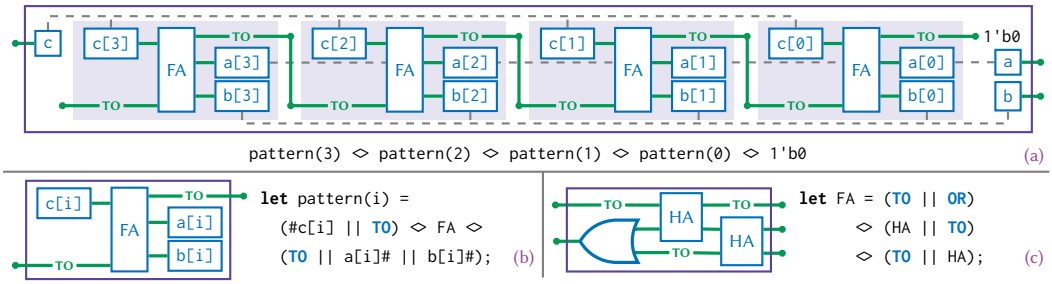


Fig. 8. Example of assembling a 4-bit ripple-carry adder, where (a) is the overview of the whole adder consisting of a repeating pattern, (b) is the pattern abstracted as a parameterized macro, and (c) shows the inner structure of the FA used in the pattern, which further composes half adders defined in Fig. 7. Note that here **TO** is a pass-through component (anonymous wire) aligning components to fit a grid, making the schematic clearer.

expressions on Line 3 and Line 4 both construct a 2-to-1 component, and composing them with || would create a 4-to-2 component, which is not our goal. To address this, # is introduced to modify the signature of an expression. As shown in Fig. 7, # after the AND gate hides its inputs, such that in the schematic, there are no corresponding lines crossing the rectangle border; thus we correctly constructed a 2-to-2 half adder. Note that wires of the same name (x1 and x2) are implicitly connected (i.e., they can be collapsed), as indicated by the dashed lines in Fig. 7.

3.2.2 Meta-Programming. To enhance usability, NetX's schematic model also extends its core with meta-programming mechanisms. In this section, we first discuss the choice of meta-programming rather than other high-level abstractions, and then provide two examples of meta-programming in NetX: parameterized macros and list comprehensions. Due to space constraints, other features (e.g., higher-order components and first-class types) are omitted but can be found in the full specification.

To select suitable high-level programming mechanisms, the key challenge here is to ensure that such mechanisms do not compromise consistency. Our insight is that since core NetX code already closely resembles circuit schematics, the patterns in schematics actually appear in the code as well, thus the mechanisms for capturing code patterns can be repurposed to capture reusable schematic patterns. This approach to capturing code patterns is known as *meta-programming*. It differs from the behavioral abstractions discussed in Section 3.1 in that: (1) The behavioral abstraction in various HDLs often has problems with consistency—even for the simplest control-flow constructs (e.g., **if-else**, `continue`, `break`), the exact circuit schematic generated from code is only determined after the time-consuming black-box synthesis, causing unpredictable or unintended circuits. (2) Meta-programming operates at the syntactic level and can be desugared through straightforward elaboration, which is transparent and unambiguous. It captures the schematic pattern reflected in code without introducing ambiguity, thus preserving consistency.

Parameterized Macro. Circuit schematics often have repeating patterns that can be captured using parameterized macros, for example, a 4-bit ripple-carry adder. It has two 4-bit inputs `a` and `b`, and produces a 4-bit sum `c`. Using `a[i]` to denote the `i`-th bit of `a`, first we compute `c[0]` by adding `a[0]` and `b[0]`. Then, to compute `c[1]`, we need not only to add `a[1]` and `b[1]`, but also to consider a carry-in from the previous computation of `c[0]`. Generally, we have the pattern: for `c[i]`, we add `a[i]`, `b[i]`, and a carry-in from the previous addition of `c[i-1]`. For the boundary cases, we set the carry-in of `c[0]` to `1'b0` (1-bit zero) to represent no carry-in, and ignore the unneeded carry-out of `c[3]`.

As shown in Fig. 8a, this pattern repeats 4 times for the 4 bits, which is reflected in the corresponding code below, where we have a chain of connected `pattern(i)` instances, each corresponding to a

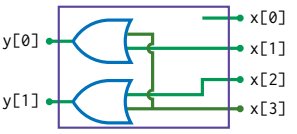
shaded box in the figure, representing the addition circuit for the i-th bit. And as shown in Fig. 8b, `pattern(i)` is a parameterized macro that binds the i-th bits of `a`, `b`, and `c` to a full adder `FA`, leaving the carry-in (right) and carry-out (left) to be connected in a chain. Here, we use the operator # mentioned in Section 3.2.1 to bind the wires: the # written before a wire hides its outputs and # written after a wire hides its inputs, thus the wires are enclosed in `pattern(i)` and invisible to the outside, making `pattern(i)` a 1-to-1 component that can be chained. Finally, the full adder `FA` is defined in Fig. 8c. It is a component that takes three 1-bit inputs (`a[i]`, `b[i]`, and a carry-in) and produces two 1-bit outputs (`c[i]` and a carry-out). It first uses a half adder `HA` (the 2-input adder defined in Fig. 7) to add `a[i]` and `b[i]`, then uses another `HA` to add the carry-in, and finally uses an OR gate to compute the carry-out.

Moreover, note that the chain of `pattern(i)` in Fig. 8a can be further generalized using recursion: `let adder(n) = if n > 0 then pattern(n) <> adder(n - 1) else 1'b0` defines a recursive macro `adder(n)` that constructs a chain of n patterns. Here, the base case is `1'b0`, and the recursive case `pattern(n) <> adder(n - 1)` connects a `pattern(n)` to the already constructed chain of $n-1$ patterns.

Notably, the code snippet here closely resembles the schematic, with each identifier corresponding to a block in the diagram, and the composition operators corresponding to the connections. In this sense, NetX code is organized like a grid-based schematic (breadboard), where: (1) components are aligned along horizontal and vertical tracks via the combinators `<>` and `||`, respectively; (2) jumper wires (e.g., `a`, `b`, `c`) are used to express more complex topologies and interconnections; and (3) anonymous wires (i.e., `TO`) are used for further alignment and clarity.

List Comprehension. Since hardware is inherently highly parallel, the parallel combinator `||` is extensively used to assemble multiple components. So we introduce the list $[e_1, \ldots, e_n]$ as syntactic sugar for $(e_1$ `||` $\cdots$ `||` $e_n)$, representing a list of parallel components. This syntactic sugar becomes powerful when combined with list comprehensions, allowing developers to concisely capture complex schematic patterns that are less apparent than simple repetition.

For example, Fig. 9 illustrates how to use lists to assemble a 2^n-to-n encoder. For simplicity, we start with a 4-to-2 encoder whose schematic is shown on the left of Fig. 9 and generalize it later. It converts a 4-bit one-hot input to a 2-bit binary output, that is: (1) It has a 4-bit one-hot input `x`, meaning that only one bit (namely, the hot bit) of `x` is `1` while the others are `0`. (2) Supposing the hot bit is `x[j]`, the 2-bit binary output `y` should be the binary encoding of `j`. For example, when the hot bit is `x[3]`, the output `y` should be `11` (binary for `3`). To implement this, consider how to determine each bit of the output `y`. `y[0]` is `1` if the rightmost bit of `j` is `1`, so we have `y[0] <> OR <> (x[1] || x[3])`. Also, `y[1]` is `1` if the next bit of `j` is `1`, so we have `y[1] <> OR <> (x[2] || x[3])`. Now consider how to generalize the pattern here for a 2^n-to-n encoder: each output bit `y[i]` is the output of an OR gate; and the gate should take all `x[j]` as inputs where the `i`-th bit of `j` is `1`. To capture this pattern, we use a list comprehension that iterates over all input indices `j` and selects those contributing to `y[i]`, as illustrated on the right of Fig. 9. In the body of the for loop, the list comprehension `[x[j] | j in [0..2**n], when (j >> i) & 1]` evaluates to a list of `x[j]` with two qualifiers: (1) `j in [0..2**n]` specifies that `j` is an integer between 0 (inclusive) and 2^n (exclusive). (2) `when (j >> i) & 1` filters the list, retaining only those `j` where the `i`-th bit of `j` is `1`. For instance, when `n == 2` and `i == 0`, it evaluates to `[x[1], x[3]]`, which is exactly what we need for `y[0]`.




```
for i in [0..n] {
  y[i] <> OR <> [
    x[j] | j in [0..2**n],
           when (j >> i) & 1];
}
```

Fig. 9. Example of assembling a 4-to-2 encoder.

List comprehensions are particularly useful when dealing with complex wiring, which is common in real-world circuits, such as the encoder shown above and others like decoders, arbiters, barrel shifters, etc. They enable developers to select subsets of a large set of wires based on certain patterns and, as noted by a senior developer in our user study (Section 5.3), are impressively useful.

4 Formalism

In this section, we introduce λ_{NX}, a core language that encapsulates the essential features of NetX. By leveraging λ_{NX}, we can establish a rigorous understanding of NetX's programming model and can reason about its properties, particularly those related to consistency.

4.1 Syntax and Type System

Syntax. A λ_{NX} program is a single expression e. Its syntax is defined below, consisting of two parts:

$e ::= x \mid e \diamond e \mid (e \mid\mid e) \mid e\# \mid \#e \mid n \mid \lambda x : \tau.\, e \mid e(e) \mid \textbf{let}\ x : \tau = e\ \textbf{in}\ e \mid e\ \text{op}\ e \mid \textbf{if}\ e\ \textbf{then}\ e\ \textbf{else}\ e$

$x ::=$ <identifier> $\quad n ::=$ <number> $\quad \tau ::=$ <type> $\quad \text{op} ::=$ <number operator>

(a) Standard simply-typed λ-calculus for meta-programming, including λ-abstraction $\lambda x : \tau.\, e$, function application $e(e)$, and recursive let-binding $\textbf{let}\ x : \tau = e\ \textbf{in}\ e$. List comprehension presented in Section 3 is not included here for simplicity; it can be desugared into λ_{NX}.

(b) Circuit expressions (in gray) that describe circuit schematics using: (1) the identifier x representing basic components (e.g., AND, OR, NOT, and REGISTER); (2) sequential composition $e_1 \diamond e_2$ connecting two components in series, where the output of e_2 becomes the input of e_1; (3) parallel composition $e_1 \mid\mid e_2$ connecting two components in parallel, allowing them to operate simultaneously; and (4) block operators $e\#$ and $\#e$ modifying the signature of a component by hiding its inputs or outputs, respectively; these two operators are used to facilitate meta-programming.

Type System. The type system of λ_{NX} also consists of two parts: (a) circuit types that dictate the legal assembly of circuit components. A circuit component is a box that takes a series of input signals and produces a series of output signals, typed as $\langle \texttt{sig} \cdots \texttt{sig} \rangle \rightsquigarrow \langle \texttt{sig} \cdots \texttt{sig} \rangle$, where `sig` can be either a bn carrying data of n-bit width, or a `clock` carrying a 1-bit synchronous clock signal. Here, clocks and 1-bit signals may be used interchangeably in rare cases, where we require an explicit component like AS_CLK : $\langle \texttt{b1} \rangle \rightsquigarrow \langle \texttt{clock} \rangle$ in Γ to perform the conversion. (b) standard types from λ-calculus, including `nat` for natural numbers and $\tau \to \tau$ for functions. Formally, the types are:

$$\tau ::= \langle \texttt{sig} \cdots \texttt{sig} \rangle \rightsquigarrow \langle \texttt{sig} \cdots \texttt{sig} \rangle \mid \texttt{nat} \mid \tau \to \tau \qquad \texttt{sig} ::= \texttt{clock} \mid \texttt{b}n, n \in \mathbb{N}^+$$

Here we define the typing judgments of λ_{NX} in Fig. 10. The notations include: (1) The typing context Γ is a finite sequence of bindings $e : \tau$ that associate expressions with their types. (2) $\Gamma \vdash e : \tau$ denotes that expression e has type τ under the context Γ; (3) $\Gamma[e \mapsto \tau]$ denotes updating the context Γ with a new binding; (4) $+\!\!+$ denotes the concatenation of two tuples, e.g., $\langle \texttt{sig}_1, \texttt{sig}_2 \rangle +\!\!+ \langle \texttt{sig}_3 \rangle$ results in $\langle \texttt{sig}_1, \texttt{sig}_2, \texttt{sig}_3 \rangle$; and (5) $\langle\rangle$ denotes the empty tuple containing no signals.

Besides the standard rules of λ-calculus, we have: (1) T-Seq requires the connecting components to have matching types, i.e., in $e_1 \diamond e_2$, the output type of e_2 should match the input type of e_1; (2) T-Par concatenates the types of the two components; (3) T-Blk-I and T-Blk-O modify the signature of an expression by setting its inputs or outputs to be empty.

$$\frac{\Gamma \vdash e_2 : a \rightsquigarrow b \qquad \Gamma \vdash e_1 : b \rightsquigarrow c}{\Gamma \vdash e_1 \diamond e_2 : a \rightsquigarrow c}\ \text{T-Seq} \qquad \frac{\Gamma \vdash e_1 : i \rightsquigarrow o \qquad \Gamma \vdash e_2 : i' \rightsquigarrow o'}{\Gamma \vdash e_1 \mid\mid e_2 : i +\!\!+ i' \rightsquigarrow o +\!\!+ o'}\ \text{T-Par}$$

$$\frac{\Gamma \vdash e : i \rightsquigarrow o}{\Gamma \vdash \#e : i \rightsquigarrow \langle\rangle}\ \text{T-Blk-O} \qquad \frac{\Gamma \vdash e : i \rightsquigarrow o}{\Gamma \vdash e\# : \langle\rangle \rightsquigarrow o}\ \text{T-Blk-I} \qquad \frac{}{\Gamma \vdash n : \texttt{nat}}\ \text{T-Num} \qquad \frac{\Gamma \vdash e_1 : \texttt{nat} \qquad \Gamma \vdash e_2 : \texttt{nat}}{\Gamma \vdash e_1\ \text{op}\ e_2 : \texttt{nat}}\ \text{T-Op}$$

$$\frac{\Gamma[x \mapsto \tau_1] \vdash e_1 : \tau_2}{\Gamma \vdash \lambda x : \tau_1.\, e_1 : \tau_1 \to \tau_2}\ \text{T-Abs} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1(e_2) : \tau_2}\ \text{T-App}$$

$$\frac{\Gamma[x \mapsto \tau_1] \vdash e_1 : \tau_1 \qquad \Gamma[x \mapsto \tau_1] \vdash e_2 : \tau_2}{\Gamma \vdash \textbf{let}\ x : \tau_1 = e_1\ \textbf{in}\ e_2 : \tau_2}\ \text{T-Let} \qquad \frac{\Gamma \vdash e_1 : \texttt{nat} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 : \tau}\ \text{T-Cond}$$

Fig. 10. Typing Judgments of λ_{NX}.

4.2 Semantics

λ_{NX} is designed to capture circuit schematics to provide consistency. However, besides circuit expressions directly aligning with schematics, λ_{NX} also includes non-circuit constructs from λ-calculus for meta-programming. Thus, λ_{NX} has a two-step semantics: (1) the elaboration of non-circuit constructs, a syntactic transformation that preserves the schematic core; and (2) the evaluation of circuit expressions, intuitively producing a graph (schematic) representing logical circuits.

Elaboration. The elaboration follows a standard β-reduction process from λ-calculus, with the distinction that the schematic core remains intact during transformation. That is, we elaborate an expression e into a normal form $\bar{e}$ containing the preserved schematic core (in gray) and standard λ-calculus normal forms. Formally, the normal form is defined as:

$$\bar{e} ::= x \mid \bar{e} \diamond \bar{e} \mid (\bar{e} \mid\mid \bar{e}) \mid \bar{e}\# \mid \#\bar{e} \mid n \mid \lambda x.\, e$$

We use call-by-value small-step elaboration $e_1 \hookrightarrow e_2$. Function application substitutes the argument into the function body, while recursive let-bindings unfold through substitution. Standard β-reduction proceeds within circuit constructs such as $\diamond$ and $||$, elaborating their operands while preserving the circuit operators themselves. Since the elaboration rules are almost standard, we omit them here for brevity and refer readers to the supplementary material for details.

Evaluation. Since λ_{NX} is designed to capture circuit schematics, a λ_{NX} program is intuitively evaluated to a circuit graph (schematic) G. For example, a λ_{NX} program of an identifier AND () is evaluated to  . Here dots are vertices and is a hyperedge (see Definition 4.1). Let $\Gamma, e \Rightarrow \mathsf{G}$ denote that e is evaluated to G in the context Γ; we define the big-step semantics in Fig. 11.

Definition 4.1 (Circuit Graph). A circuit graph $\mathsf{G} = (\mathsf{V}, \mathsf{E}, \mathsf{I}, \mathsf{O})$ is a labeled directed hypergraph. Here, V is the set of vertices, with each vertex representing a signal in the circuit; E is the set of hyperedges, with each hyperedge representing a component that connects a series of input vertices to a series of output vertices; and I and O are the ordered lists of input and output vertices, respectively, representing the interface of the circuit. Notably, each vertex is formally indexed with a unique number and annotated with a type. Formally, $\mathsf{V} \subset \{n : \mathtt{sig} \mid n \in \mathbb{N}\}$, $\mathsf{E} : \mathsf{V}^* \times \mathsf{V}^* \to \ell$, and $\mathsf{I}, \mathsf{O} \in \mathsf{V}^*$ where $\mathsf{V}^* ::= \bigcup_{n\in\mathbb{N}} \mathsf{V}^n$ and ℓ denotes labels. For example, the graph is written as:

$$\mathsf{V} = \{1 : \mathsf{b1}, 2 : \mathsf{b1}, 3 : \mathsf{b1}\} \quad \mathsf{E} = (\langle 1 : \mathsf{b1}, 2 : \mathsf{b1}\rangle, \langle 3 : \mathsf{b1}\rangle) \mapsto \mathsf{AND} \quad \mathsf{I} = \langle 1 : \mathsf{b1}, 2 : \mathsf{b1}\rangle \quad \mathsf{O} = \langle 3 : \mathsf{b1}\rangle$$

Example 4.2. To understand the rules in Fig. 11, consider the 4-step evaluation of NOT $\diamond$ AND: (1) As shown in Fig. 12a, given an identifier AND of type $\langle \mathsf{b1}, \mathsf{b1}\rangle \rightsquigarrow \langle \mathsf{b1}\rangle$, by G-Comp, it is evaluated to a graph (namely G_{AND}) with two input vertices and one output vertex. (2) Similarly, in Fig. 12b, we evaluate NOT to a graph G_{NOT}. (3) When composing the two graphs, there is a conflict since the vertices of both graphs are indexed from 1. Therefore, we *re-index* the vertices in G_{NOT}, i.e., we add the number of vertices in G_{AND} (i.e., 3, denoted as $|\mathsf{V}_{\text{AND}}|$) to the index of all vertices in

$$\frac{\Gamma \vdash x : \langle i_1 \cdots i_n\rangle \rightsquigarrow \langle o_1 \cdots o_m\rangle \quad \mathsf{I} = \langle (1 : i_1) \cdots (n : i_n)\rangle \quad \mathsf{O} = \langle (n+1 : o_1) \cdots (n+m : o_m)\rangle}{\Gamma, x \Rightarrow (\mathsf{I} +\!\!+ \mathsf{O}, \langle \mathsf{I}, \mathsf{O}\rangle \mapsto x, \mathsf{I}, \mathsf{O})}\ \text{G-Comp} \qquad \frac{\Gamma, e \Rightarrow (\mathsf{V}, \mathsf{E}, \mathsf{I}, \mathsf{O})}{\Gamma, \#e \Rightarrow (\mathsf{V}, \mathsf{E}, \mathsf{I}, \emptyset)}\ \text{G-Blk-O}$$

$$\frac{\Gamma, e_1 \Rightarrow (\mathsf{V}_1, \mathsf{E}_1, \mathsf{I}_1, \mathsf{O}_1) \quad \Gamma, e_2 \Rightarrow \mathsf{G}_2 \quad \mathsf{G}_2 \rhd |\mathsf{V}_1| = (\mathsf{V}_2, \mathsf{E}_2, \mathsf{I}_2, \mathsf{O}_2)}{\Gamma, e_1 \mid\mid e_2 \Rightarrow (\mathsf{V}_1 +\!\!+ \mathsf{V}_2, \mathsf{E}_1 \cup \mathsf{E}_2, \mathsf{I}_1 +\!\!+ \mathsf{I}_2, \mathsf{O}_1 +\!\!+ \mathsf{O}_2)}\ \text{G-Par} \qquad \frac{\Gamma, e \Rightarrow (\mathsf{V}, \mathsf{E}, \mathsf{I}, \mathsf{O})}{\Gamma, e\# \Rightarrow (\mathsf{V}, \mathsf{E}, \emptyset, \mathsf{O})}\ \text{G-Blk-I}$$

$$\frac{\Gamma, e_2 \Rightarrow (\mathsf{V}_2, \mathsf{E}_2, \mathsf{I}, \langle x_1 \cdots x_k\rangle) \quad \Gamma, e_1 \Rightarrow \mathsf{G}_1 \quad \mathsf{G}_1 \rhd |\mathsf{V}_2| = (\mathsf{V}_1, \mathsf{E}_1, \langle y_1 \cdots y_k\rangle, \mathsf{O})}{\Gamma, e_1 \diamond e_2 \Rightarrow (\mathsf{V}_1 +\!\!+ \mathsf{V}_2, (\mathsf{E}_1 \cup \mathsf{E}_2)[\langle x_1, y_1\rangle \mapsto \alpha] \cdots [\langle x_k, y_k\rangle \mapsto \alpha], \mathsf{I}, \mathsf{O})}\ \text{G-Seq}$$

Fig. 11. Evaluation Rules for λ_{NX}.

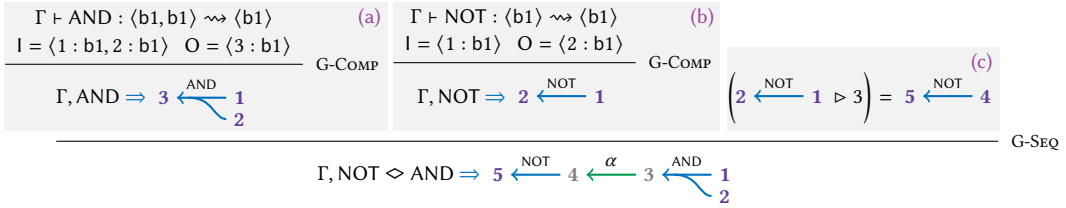
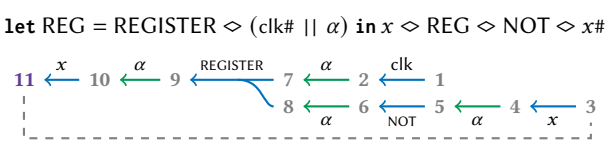


Fig. 12. Proof tree of evaluating NOT ⋄ AND to a circuit graph. In circuit graphs, numbers denote vertices (with input and output vertices highlighted in purple), and arrows denote hyperedges (components).

G_{NOT}, resulting in a new graph G'_{NOT} whose vertices are re-indexed to 4 and 5, as shown in Fig. 12c, denoted as $G_{NOT} \triangleright |V_{AND}|$. (4) Finally, we connect the output of G_{AND} to the input of G'_{NOT} with anonymous wires (denoted as α, corresponding to the `TO` in Section 3.2) using G-Seq.

Example 4.3 (Sequential Circuits). To describe sequential (i.e., stateful) circuits in NetX, users can compose the primitive stateful component REGISTER : ⟨clock, b1⟩ ⇝ ⟨b1⟩ with other combinational components. For example, consider the following λ_{NX} program constructing a sequential circuit.



The macro REG defines a register that synchronizes on the clock signal clk. We use a named wire x to construct a feedback loop: at each clock cycle, we read the value of x, negate it, and write it back to the register as the new value of x for the next cycle. The design of NetX allows users to move the register along circuit paths easily, e.g., by swapping the positions of REG and NOT. It facilitates various optimizations, such as pipelining and retiming that are widely used in actual hardware design involving more complicated sequential compositions.

Remark 4.4 (Simulation Semantics). We can interpret the circuit graph to simulate the circuit behavior in a software environment. Due to space constraints, we do not provide full specifications of the simulation semantics here and only provide the key idea: we treat a circuit as a reactive system where a value update propagates through the circuit graph, which is much simpler than the simulation semantics of behavioral HDLs [13, 56]. See the supplementary material for more details.

Note that the serial and parallel combinators ⋄ and || are widely used in systems such as process calculi and category theory [27, 37, 44, 60, 72]. In these systems, they correspond to algebraic composition (∘) and product (×) over maps, functions, or morphisms. In NetX, however, they are interpreted schematically rather than algebraically, which helps avoid abstractions that could undermine consistency (Section 3.1). This reflects the less-is-more design principle adopted by NetX: to manages complexity and preserving consistency using a deliberately small set of language constructs.

4.3 Discussions on Consistency

In previous sections, we provided intuition on λ_{NX}'s code-to-circuit consistency through figures and examples (e.g., Fig. 6-8 and Example 4.2). This consistency is captured syntactically: (1) circuit expressions form the schematic core, which is a grid where components are aligned horizontally and vertically by ⋄ and ||, and more complex topologies are represented by names as jumper wires. (2) meta-programming does not break consistency since it also operates syntactically: they can be eliminated by straightforward elaboration and preserves the structure of circuit graphs. As a result, schematic properties of the generated graph can be reasoned about directly from the source.

Below, we discuss some properties λ_{NX} can guarantee or reason about at source-level thanks to its consistent design: (1) Combinational loops, a common hardware bug that is often handled by testing or verification [15, 33, 58] in existing languages; (2) Circuit planarity, a property desired for

better physical layout that is underexplored for source languages [69]; (3) locality that establishes correspondence between code subexpression and circuit subgraph, which is broken for languages containing behavioral structures like `if-else`; and (4) IO consistency that ensures the generated circuit should match the IO types in code that is mostly weak and implicit in many languages [3, 39, 68].

Combinational Loops. As a common bug in hardware design, combinational loops are cycles containing no REGISTERs in a circuit graph. For instance, $x \diamond \text{ADD} \diamond (x \mid\mid y)$ forms a loop, requiring the value of x to be equal to $x + y$, which is logically impossible when y is non-zero, resulting in unexpected physical voltages. Below, we show how reasoning about combinational loops is (a) intuitive in λ_{NX}'s schematic programming model but (b) complex in behavioral models.

(a) In λ_{NX}, not using a name twice in a chain of $\diamond$ guarantees no loops since: (1) *loops can only be introduced by names.* That is, without named wires, the indexing rules of λ_{NX} guarantee that hyperedges always point from smaller-indexed vertices to larger-indexed vertices (Theorem 4.5), making the circuit graph acyclic (Corollary 4.6). (2) *when names introduce loops, there must be a name that occurs twice in a chain of* $\diamond$. That is, as illustrated below, to construct a cycle in the acyclic graph, the extra *implicit connectivity* (the dashed orange line) introduced by named wires must intersect with an existing path, i.e., a chain of $\diamond$.



Theorem 4.5 (G-Ascending). $\forall \Gamma, e, \mathsf{G}.\ (\Gamma, e \Rightarrow \mathsf{G}) \Rightarrow (\forall \langle i, o \rangle \in \mathrm{dom}(\mathsf{E}).\ \forall u \in i.\ \forall v \in o.\ u < v)$.

Corollary 4.6. *The graph* G *is acyclic since its vertex indices induce a valid topological order.*

Thus, in λ_{NX}, determining whether a circuit contains a combinational loop reduces to finding a name that occurs twice in a chain of $\diamond$ containing no REGISTERs. So in the compiler, we can treat REGISTERs as non-connected edges and check for repeated names in each connected chain of $\diamond$ to detect combinational loops, which is guaranteed to be correct by Corollary 4.6.

(b) However, reasoning about loops in other HDLs requires more care with behavioral constructs (e.g., control flow and concurrency); this is why previous works [15, 33] require more complex reasoning. For example, for the Verilog code `x = 0; for (int i = 0; i < 4; i++) x = x + a[i]`, though `x` appears on both sides of an assignment (similar to a name appearing on both sides of $\diamond$ in λ_{NX}), the code actually does not form a combinational loop since after unrolling and inlining, it becomes `x = 0 + a[0] + a[1] + a[2] + a[3]`.

Circuit Planarity. Planarity means that a circuit can be drawn on a plane without edge crossings; it is often desired for easier physical layout and better performance [69]. Similar to combinational loops, when not considering named wires, λ_{NX} guarantees planarity of the circuit graph (Theorem 4.7). Further, when considering named wires, this theorem makes it possible to reason about planarity incrementally: for each named wire introducing a new implicit edge, we can perform the Hopcroft-Tarjan incremental planarity test [48], which is efficient and provides precise feedback on which exact wire breaks planarity. Moreover, since all names are local to their lexical scope in NetX, the scale of the graph affected by each name is limited, further reducing the verification overhead, whereas, it is generally unclear how to reason about planarity in behavioral HDLs as far as we know.

Theorem 4.7 (G-Planarity). $\forall \Gamma, e, \mathsf{G}.\ (\Gamma, e \Rightarrow \mathsf{G}) \Rightarrow \mathsf{G}$ *is planar, i.e., its vertices and hyperedges admit an embedding in the Euclidean plane such that no two connecting segments intersect.*

Remark 4.8 (Names and Circuit Topology). Above we showed an important functionality of named wires: expressing complex topologies. Although circuits are always acyclic and planar without named wires (Corollary 4.6 and Theorem 4.7), real-world circuits can have cycles (e.g., feedback loops in sequential circuits) and non-planar structures (e.g., multi-layer PCBs and 3D integrated circuits). Here, named wires can effectively represent these topologies and help to manage their complexity; otherwise, introducing extra operators to model them [44, 78] can be cumbersome.

Local Correspondence. We can establish the correspondence that the subexpression relationship in λ_{NX} source code should be preserved as the subgraph relationship in the generated circuit graphs. This is a structural consistency between code and schematics which guarantees that local source-code reasoning can be generalized to the whole program schematic:

Theorem 4.9 (Local-Correspondence). *For a whole execution* $e \hookrightarrow^* \bar{e} \Rightarrow \mathrm{G}$, *each subexpression of* e *yields* $n \in \mathbb{N}$ *subexpressions* $\bar{e}_1, \ldots, \bar{e}_n$ *of* $\bar{e}$ *and for each* $\bar{e}_i \Rightarrow \mathrm{G}_i$, G_i *is a subgraph of* G.

IO Consistency. Types, as a classical reasoning tool, also have their unique value in the schematic model. In λ_{NX}, types dictate valid inputs and outputs (IO) of all expressions. Thus, type soundness of λ_{NX} ensures the IO of evaluated circuits is consistent with the type of λ_{NX} programs. In contrast, in behavioral HDLs, there are various problems with IO consistency: (1) not all program constructs can be typed, such as processes in VHDL [40]; (2) types can be non-strict, e.g., a port declared as an output is allowed to be used as an input, see Page 179 of the Verilog standard [39]; (3) types can be implicit, causing confusion and inconsistency, such as the known issue for `Flipped` in Chisel [83].

Theorems that guarantee IO consistency, such as the preservation and progress theorems of elaboration and evaluation, and supporting theorems like confluence and type uniqueness, are left to the supplementary material due to space constraints.

5 Implementation and Evaluation

Alongside the Rocq mechanization of NetX's core calculus and properties (~15,000 lines), we implement the NetX toolchain (~30,000 lines) containing various development, simulation, and debugging tools. The toolchain is validated by implementing 12 diverse designs totaling 5,000+ lines of NetX code, forming a NetX benchmark suite (Section 5.1). We further evaluate NetX through a case study (Section 5.2) and a preliminary user study (Section 5.3) based on the benchmark suite.

AI-Use. ChatGPT and DeepSeek assisted with only ~ 5% of the implementation and mechanization. The authors reviewed the AI-assisted contributions and are responsible for all the results.

5.1 Toolchain

Fig. 13 shows the architecture of the NetX toolchain, organized around the Nxon (pronounced as *nek-sawn*) circuit-graph representation, which is a Json-based low-level schematic representation. The compiler translates source code into Nxon, which supports backend circuit generation, visualization, and simulation. The package manager and language server protocol (LSP) further assist source code development. Later we will show the usecse of the toochain in Section 5.2.

We implemented the NetX compiler in over 20,000 lines of Java code. The compiler first translates NetX source code into Nxon and further into various backend representations. During compilation, we parse and elaborate the code, as well as perform multipass analysis that not only reports hardware bugs (e.g., multiple drivers [13] and combinational loops [15], as formalized in Section 4.3), but also performs optimizations such as duplication elimination, bit operation simplification, and alias

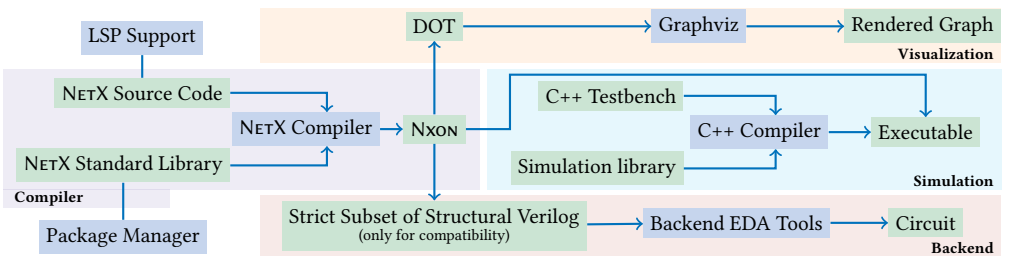


Fig. 13. Overview of the NetX toolchain.

analysis. For instance, the alias analysis collapses redundant vertices in the circuit graph by finding the transitive closure of aliases (the anonymous wires labeled with α, see Section 4). Notably, these optimizations do not break the consistent design of NetX; for example, the graph optimized by the alias analysis can be guaranteed to be homomorphic to the original one [36].

For simulation, we interpret the generated Nxon circuit graph, with stimuli and assertions written in C++. This separates schematic circuit descriptions from behavioral test code while enabling integration with existing C++ debugging and testing tools. It also supports waveform dumping, simulation with external IP models, and other features omitted here for brevity.

For backend code generation, the consistency-critical step is Nxon generation, defined by the schematic semantics of NetX and producing an explicit circuit graph. In principle, this graph can be translated directly to backend netlist formats while preserving structure. In practice, however, such formats are often tool-specific, so we use structural Verilog as an intermediate format to interface with existing toolchains (e.g., the FPGA platform toolkit used in our experiment). Structural Verilog is used only for tool compatibility; consistency is established at the Nxon level. Although structural Verilog may have semantic pitfalls if interpreted behaviorally (Section 3.1), our backend emits only a restricted subset of structural Verilog that, in our experience, mainstream backend tools handle reliably. Nxon can also be translated to DOT for visualization, with other targets left for future work.

Benchmarks. To demonstrate the feasibility of using NetX for practical hardware designs, we implemented 12 benchmarks in NetX, totaling 5,000+ lines of code, including a computer system built around a five-stage pipelined RV32I CPU in approximately 2,000 lines. We choose benchmarks from representative textbooks, papers, and projects [9, 52, 57, 64, 67], spanning video processing, cryptography, networking, interconnects, and computer architecture. It exercises representative design patterns, such as pipelines (CPU), irregular wiring (CAVLC), finite state machines (DDR2), iterative datapaths (SQRT), and crossbar-based interconnects (AXI). Although moderate in scale compared to industrial designs, these benchmarks demonstrate that NetX can express a range of practical hardware structures. The complete benchmark overview is provided in the supplementary material.

5.2 Case Study

To evaluate how NetX and its toolchain can be used to implement, simulate, and debug large systems, we report a case study of building a computer system on an FPGA. The system runs a read-eval-print loop (REPL) for Scheme, allowing users to interact with the CPU through a keyboard, a monitor, and on-board peripherals (Fig. 14). It serves as a fair stress test for NetX: IO devices with diverse interfaces and characteristics are integrated; and the 5-stage pipelined CPU is fairly complex, featuring hazard detection, pipeline forwarding, and branch target buffers (BTB) for prediction.

Below, we report the experience and insights gained from implementing, simulating, and debugging this system.



Fig. 14. The system running a Scheme REPL on an FPGA implemented in NetX.

Implementation. NetX is capable of describing high-level design patterns concisely. For example, finite state machines (FSMs) are often recognized as a high-level pattern that might be hard to describe in some low-level languages; NetX can effectively describe FSMs with good readability.

An FSM is a circuit consisting of a sequential register holding the current state and the combinational logic deciding the next state. In traditional HDLs, there are two ways to describe FSMs: the two-process style decoupling the sequential and combinational parts, and the one-process style mixing them together. NetX naturally supports the two-process style since the sequential and

combinational parts are schematically separated in real circuits. The one-process style, although very expressive, is essentially behavioral and rejected by NetX. Given that prior literature [19] and our study (Section 2.3.3) both suggest that the two-process style is generally recommended for better readability and maintainability, NetX seems to be well-aligned with the recommended practice.

```
1  import std.selector.MUX;
2  enum VGA_H_STATE {H_VISIBLE, H_FRONT, H_SYNC, H_BACK}
3
4  wire h_state of VGA_H_STATE;
5  h_state <> REGISTER(pos_edge=true) <> [vga_clk,
6      MUX(1) <> [h_zero, h_state,
7          MUX(2) <> {
8              sel : h_state,
9              case[H_VISIBLE] : H_FRONT,
10             case[H_FRONT]   : H_SYNC,
11             case[H_SYNC]    : H_BACK,
12             case[H_BACK]    : H_VISIBLE  }
13     ]
14 ];
```

For example, the code on the left taken from the VGA controller describes an FSM. Here, the sequential part (Lines 4–6) describing how and when the state register should be updated; the combinational part (Lines 7–13) deciding the next state are syntactically decoupled. Here, we introduce some new syntax: Line 1 imports the multiplexer from the standard library, and Line 2 declares the enumeration of all the states; the compiler may assign bit width and encoding to the enumeration if not specified. The imported multiplexer has the signature `MUX(n): [sel, case[0], ..., case[2**n - 1]] → out`, outputting `case[i]` when `sel` equals `i` (`n` is the bit width of `sel`). When using `MUX`, developers can connect ports both by order and by name; the former is concise (Line 6), while the latter is more readable (Line 7).

Simulation. We use a software language (C++) to specify simulation stimuli for flexibility and tool reuse, as discussed in Section 5.1. For example, the code snippet on the right shows the C++ stimuli for the CPU. Here, Lines 1–2 initialize the simulation context by reading the circuit from the file `circuit.nxon` and loading the program file `instr.hex` into the memory; here, `Memory` is a native model in C++ of the on-board IP core whose implementation is unavailable for simulation. Lines 5–6 read the program counter, fetching instructions from the memory, and feed them back to the circuit. Line 8 flips the clock signal twice to advance the time by one cycle. Line 10 finally checks whether the value of the `x10` register matches the expected `0x00C0FFEE` after executing the last instruction (`0xDEAD10CC`); if so, the test passes, otherwise it fails, as specified by the RISC-V test suite [76]. By setting a breakpoint at Line 5, we can debug the CPU cycle by cycle and inspect each loaded instruction; we can also set breakpoints to inspect every memory access, pipeline flush, etc., to debug with fine-grained datapaths. Such breakpoints can be set directly in a C++ debugger, which is often less straightforward in traditional workflows.

```
1  auto ctx = parse_circuit("circuit.nxon");
2  auto mem = new Memory("instr.hex");
3  auto instr = 0;
4  while (instr != 0xDEAD10CC) {
5    instr = mem→read_word(ctx.get("PC"));
6    ctx.set("instr", instr);
7    /* Other stimuli */
8    ctx.flip("clk"); ctx.flip("clk");
9  }
10 assert(ctx.get("x10") == 0x00C0FFEE);
```

Debugging. Timing closure is the process to iteratively adjust a design to meet timing constraints, where NetX's consistent design becomes useful; for example, under the constraint requiring clock frequency to reach 50 MHz, each circuit path must have a delay below 20 ns. Backend tools report the endpoints of paths whose delays exceed the limit and developers must recover the source paths and revise them. Recovering the path between endpoints is challenging in behavioral HDLs: even when endpoints map to source wires, the path may traverse control flow, process boundaries, and implicit compiler-generated logic. Predicting how source modifications affect that path can also be difficult.

By contrast, in NetX, the corresponding source path is an explicit chain of `<>` connections, making structural modifications predictable. Concretely, during timing closure, the backend reported a 31 ns path from `data_mem|ram_block1a47~portb_address_reg0` to `CPU|_19`, exceeding the 20 ns budget. NetX's preserved source location maps `_19` to `rv32i.nx:35:11:ifid.instr`, identifying wire `ifid.instr` at line 35, column 11 of `rv32i.nx`. From there, we identified the `<>` chain from data memory through `exmem_fwd`, `ALU`, `BRANCH_COND`, and `redirect_flush` to `ifid.instr`. This exposed an aggressive forwarding path from

data memory to the ALU; replacing it with a more conservative path is therefore an easy fix. We then addressed subtler violations, such as rebalancing a multiplexer hierarchy, adjusting pipeline register placement to break long critical paths, etc., all of which were straightforward due to consistent design (see the supplementary material for examples). In contrast, comparable Verilog changes would restructure behavioral descriptions that are difficult to predict (Section 3.1). The system ultimately reached a theoretical frequency of 58.56 MHz, exceeding the 50 MHz ceiling of the FPGA.

Comparable Performance to Verilog. Following prior work [61] comparing two HDLs through a single benchmark, we compare the performance of NetX and Verilog on two benchmarks, a CPU and an FFT component. The Verilog versions were carefully reimplemented based on the NetX versions. The results suggest NetX has no substantial performance overhead in various metrics compared to Verilog. Across the two designs, NetX achieves approximately 8–9% higher frequency and uses 5% fewer logic elements, with slightly higher estimated power consumption. Code size and simulation time are comparable; see the supplementary material for details.

5.3 Preliminary User Study

We further collected qualitative feedback on NetX's design and toolchain from eight professional hardware developers, including five participants from the empirical study (P06, P11, P13, P23, and P24) and three colleagues of previous participants (X01–X03). The substantial effort required by this study and professional developers' limited availability constrained recruitment to eight participants. Participants were not informed of our hypothesis about consistency and had no conflict of interest.

The detailed study protocol and results are provided in the supplementary material; here, we summarize the key aspects. All participants were *required* to review the tutorial, documentation, and source code of the 12 benchmarks implemented in NetX, although this review required a substantial investment of time and effort—it formed an important part of the evaluation: drawing on their day-to-day hardware-development experience, participants could examine how NetX's constructs were applied across diverse designs and assess the language's design quality. To complement this systematic code-based assessment with hands-on experience, we further encouraged participants to develop projects in NetX. As a result, 6/8 participants implemented 7 projects in NetX.

Participants then completed a questionnaire about NetX's advantages and disadvantages relative to HDLs they were familiar with. With high confidence, all 8 participants identified consistency as an advantage of NetX, highlighting different aspects in their detailed comments: (1) NetX conveys design intentions directly: "Well-structured NetX code is self-documenting, directly conveying design intentions. It aligns with the intended schematic, which is never the case for other languages like Verilog." (X02); and "Traditional HDLs describe circuits through (behavioral) logic, while NetX describes circuits themselves directly." (P06). (2) NetX makes circuit design more predictable: "NetX can effectively reduce the possibility that a synthesizer misinterprets the design and generates unexpected logic." (P11); and "It is interesting that people can transcribe circuit schematic into NetX code; this correspondence can guide the beginners to better understand how to design a circuit." (P13).

Other advantages of NetX were also noted, such as the expressiveness of NetX's meta features. Disadvantages like ecosystem maturity were also raised. See the supplementary material for details.

6 Related Work

In this section, we review existing HDLs, discussing the scenarios for which these languages are suitable and how they relate to the consistency-oriented design of NetX. NetX is intended to complement existing HDLs: when consistency is the primary concern, NetX can be the primary choice of HDL; when only critical components require extra consistency, they can be rewritten in NetX and integrated with components written in other HDLs; and when other goals are prioritized, NetX may serve as a consistent compiler IR for higher-level HDLs to yield predictable compilation results.

Behavioral HDLs. Behavioral HDLs such as Verilog [39] and VHDL [40] remain the most widely used and the most mature languages in industrial practice. Their behavioral style is also inherited by closely related languages, including SystemVerilog [41], Bluespec SystemVerilog [63], and Veryl [35]; these languages further extend Verilog with various features such as object-oriented support and atomic primitives, providing a better coding experience and more powerful abstractions, yet still remain in the scope of process-based behavioral modeling. Recent languages follow the same path, embedding in modern software languages to leverage even more expressiveness, starting from Verischemelog and HML [42, 90] to Chisel, Amaranth, Clash, DFiant, PEak, PyRTL, Verik, ShakeFlow, and HDLRuby [1–4, 20, 27, 34, 66, 84]. These languages are flexible for agile hardware design, i.e., for rapid prototyping and iterating on a project; however, their behavioral constructs and abstractions can make the circuit schematic less explicit, causing inconsistency concerns. That said, there have been efforts to reduce inconsistency through compiler patches [12, 43], static scheduling strategies [7], tailored semantics [56], and the coding guidelines [19, 77]; they address particular sources of inconsistency, whereas NetX makes schematic consistency a first-class language design principle.

Functional and structural HDLs. Lava [72], Arrows [8], Clash [2], and Ruby [78] are functional languages that make circuit composition explicit and support common functional programming abstractions like pattern matching and higher-order functions. However, explicit composition does not by itself ensure schematic consistency: abstractions can still hide structural differences, as shown in Section 3.1. NetX instead restricts circuit construction to schematic operations and uses curated meta-programming abstractions that do not break schematic consistency, providing a structural basis for consistency (Theorem 4.9). Spade [74] emphasizes zero-cost abstractions, focusing on how circuit abstractions may affect circuit performance, yet does not address the schematic consistency regarding finer-grained circuit characteristics. Circuit-cj [44] builds on string diagrams for digital circuits [29, 30]; such modeling is suited for reasoning about algebraic properties, but needs additional effort manipulating monoidal categories to express complex topologies (Remark 4.8) and are not designed for the schematic properties and consistency that NetX emphasizes.

High-Level Synthesis Languages. High-level synthesis (HLS) languages take still another direction by emphasizing algorithmic description and quick deployment. Languages such as SystemC, Lime, Floh, Spatial, Lift, etc., [21, 31, 38, 45, 47, 50, 82] intentionally omit many schematic or structural details to provide a faster and smoother design experience for domain-specific hardware (e.g., accelerators for neural networks); that is, schematic consistency is not a primary concern in HLS.

Other Languages and Representations. Other works focus on specific goals; they are inspiring and might be integrated with NetX, but are less relevant to our work. Caisson [51], SecVerilog [91], and QIF-Verilog [32] study security-specific properties; Filament [61, 62] emphasizes pipeline composition and timing reasoning; Kami [14] supports formal verification in proof-oriented settings; Tydi-lang [81] focuses on typed stream interfaces; and Memo [73] emphasizes fungible memory abstractions. Polymorphic Blocks [53] is a graphical board-level design toolkit for novices. LLHD [71] is a multi-level IR for hardware compilation, into which NetX could be integrated in future work.

7 Conclusion

This paper presents NetX, a language that provides consistency through a schematic programming model. Our empirical study identifies consistency as a key practical requirement and reveals a fundamental mismatch: hardware is schematic, while common HDLs use behavioral programming models. NetX addresses this by adopting a schematic model where code is arranged to resemble circuit schematics, enabling suitable abstractions and formal guarantees for schematic consistency. We implement NetX's toolchain and evaluate it through hardware designs, a case study, and a user study. Our results indicate that NetX represents a solid step toward consistent hardware design.

Data-Availability Statement

We will open-source the NetX toolchain, the Rocq mechanization, the language specification, and the benchmarks used in our evaluation. We will also release the study protocols, questionnaires, anonymized interview records supporting the empirical study (Section 2) and user study (Section 5.3). We will submit the complete artifact for artifact evaluation; the linked record provides the preliminary anonymous artifact. The artifact is available at https://doi.org/10.5281/zenodo.23410788.

References

[1] Amaranth. 2018. Amaranth HDL: A modern hardware definition language and toolchain based on Python. https://github.com/amaranth-lang

[2] C. P. R. Baaij. 2015. *Digital circuit in CλaSH: functional specifications and type-directed synthesis*. Ph. D. Dissertation. University of Twente, Enschede.

[3] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avižienis, John Wawrzynek, and Krste Asanović. 2012. Chisel: Constructing Hardware in a Scala Embedded Language. In *Proceedings of the 49th Annual Design Automation Conference*. ACM, San Francisco California, 1216–1225. ISBN 978-1-4503-1199-1. doi:10.1145/2228360.2228584

[4] Rick Bahr, Clark Barrett, Nikhil Bhagdikar, Alex Carsello, Ross Daly, Caleb Donovick, David Durst, Kayvon Fatahalian, Kathleen Feng, Pat Hanrahan, Teguh Hofstee, Mark Horowitz, Dillon Huff, Fredrik Kjolstad, Taeyoung Kong, Qiaoyi Liu, Makai Mann, Jackson Melchert, Ankita Nayak, Aina Niemetz, Gedeon Nyengele, Priyanka Raina, Stephen Richardson, Raj Setaluri, Jeff Setter, Kavya Sreedhar, Maxwell Strange, James Thomas, Christopher Torng, Leonard Truong, Nestan Tsiskaridze, and Keyi Zhang. 2020. Creating an Agile Hardware Design Flow. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, San Francisco, CA, USA, 1–6. doi:10.1109/DAC18072.2020.9218553

[5] Armin Biere. 2007. The AIGER and-inverter graph (AIG) format version 20071012.

[6] Tracey Booth, Simone Stumpf, Jon Bird, and Sara Jones. 2016. Crossed Wires: Investigating the Problems of End-User Developers in a Physical Computing Task. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems (CHI '16)*. Association for Computing Machinery, New York, NY, USA, 3485–3497. ISBN 978-1-4503-3362-7. doi:10.1145/2858036.2858533

[7] Thomas Bourgeat, Clément Pit-Claudel, Adam Chlipala, and Arvind. 2020. The Essence of Bluespec: A Core Language for Rule-Based Hardware Design. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, London UK, 243–257. ISBN 978-1-4503-7613-6. doi:10.1145/3385412.3385965

[8] Matthias Brettschneider and Tobias Häberlein. 2013. From Arrows to Netlists Describing Hardware. In *Computational Science and Its Applications – ICCSA 2013* (Berlin, Heidelberg, 2013), Beniamino Murgante, Sanjay Misra, Maurizio Carlini, Carmelo M. Torre, Hong-Quang Nguyen, David Taniar, Bernady O. Apduhan, and Osvaldo Gervasi (Eds.). Springer, 128–143. ISBN 978-3-642-39646-5. doi:10.1007/978-3-642-39646-5_10

[9] Stephen D. Brown and Zvonko G. Vranesic. 2014. *Fundamentals of Digital Logic with Verilog Design* (third edition ed.). McGraw-Hill Higher Education, New York. ISBN 978-0-07-338054-4.

[10] Brad Burres, Dan Daly, Mark Debbage, Eliel Louzoun, Christine Severns-Williams, Naru Sundar, Nadav Turbovich, Barry Wolford, and Yadong Li. 2021. Intel's Hyperscale-Ready Infrastructure Processing Unit (IPU). In *2021 IEEE Hot Chips 33 Symposium (HCS)* (2021-08). IEEE, Palo Alto, CA, USA, 1–16. doi:10.1109/HCS52781.2021.9567455

[11] Idan Burstein. 2021. Nvidia Data Center Processing Unit (DPU) Architecture. In *2021 IEEE Hot Chips 33 Symposium (HCS)* (2021-08). IEEE, Palo Alto, CA, USA, 1–20. doi:10.1109/HCS52781.2021.9567066

[12] Vikas Chauhan, Neel Gala, and V. Kamakoti. 2016. ChADD: An ADD Based Chisel Compiler with Reduced Syntactic Variance. In *2016 29th International Conference on VLSI Design and 2016 15th International Conference on Embedded Systems (VLSID)*. IEEE, Kolkata, India, 499–504. doi:10.1109/VLSID.2016.44

[13] Qinlin Chen, Nairen Zhang, Jinpeng Wang, Tian Tan, Chang Xu, Xiaoxing Ma, and Yue Li. 2023. The Essence of Verilog: A Tractable and Tested Operational Semantics for Verilog. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2 (Oct. 2023), 234–263. doi:10.1145/3622805

[14] Joonwon Choi, Muralidaran Vijayaraghavan, Benjamin Sherman, Adam Chlipala, and Arvind. 2017. Kami: A Platform for High-Level Parametric Hardware Specification and Its Modular Verification. *Proceedings of the ACM on Programming Languages* 1, ICFP (Aug. 2017), 24:1–24:30. doi:10.1145/3110268

[15] Michael Christensen, Timothy Sherwood, Jonathan Balkind, and Ben Hardekopf. 2021. Wire Sorts: A Language Abstraction for Safe Hardware Composition. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 175–189. ISBN 978-1-4503-8391-2. doi:10.1145/3453483.3454037

[16] Celio Christopher. 2018. *A Highly Productive Implementation of an Out-of-Order Processor Generator*. Ph. D. Dissertation. University of California at Berkeley. https://escholarship.org/uc/item/5vh084rf

[17] Intel Corporation. 2024. Quartus Prime Software. https://www.intel.com/content/www/us/en/products/details/fpga/development-tools/quartus-prime.html

[18] Philippe Coussy, Daniel D. Gajski, Michael Meredith, and Andres Takach. 2009. An Introduction to High-Level Synthesis. *IEEE Design & Test of Computers* 26, 4 (July 2009), 8–17. doi:10.1109/MDT.2009.69

[19] Clifford E Cummings. 1998. State machine coding styles for synthesis. In *SNUG'98 (Synopsys Users Group San Jose, CA, 1998) Proceedings*.

[20] Deeksha Dangwal, Georgios Tzimpragos, and Timothy Sherwood. 2020. Agile Hardware Development and Instrumentation With PyRTL. *IEEE Micro* 40, 4 (July 2020), 76–84. doi:10.1109/MM.2020.2997704

[21] Stephen A. Edwards, Richard Townsend, Martha Barker, and Martha A. Kim. 2019. Compositional Dataflow Circuits. *ACM Trans. Embed. Comput. Syst.* 18, 1 (2019), 5:1–5:27. doi:10.1145/3274280

[22] Matthias Felleisen and Robert Hieb. 1992. The Revised Report on the Syntactic Theories of Sequential Control and State. *Theoretical Computer Science* 103, 2 (1992), 235–271. doi:10.1016/0304-3975(92)90014-7

[23] Peter Flake, Phil Moorby, Steve Golson, Arturo Salz, and Simon Davidmann. 2020. Verilog HDL and Its Ancestors and Descendants. *Proc. ACM Program. Lang.* 4 (2020), 87:1–87:90. Issue HOPL. doi:10.1145/3386337

[24] Harry Foster. 2025. *FPGA Functional Verification Trend Report - 2024*. Siemens Verification Academy. https://verificationacademy.com/resource/0021d316-9b98-4857-834a-e40f96279fbe

[25] Harry Foster. 2025. *IC/ASIC Functional Verification Trend Report - 2024*. Siemens Verification Academy. https://verificationacademy.com/resource/a661652a-b077-4b68-98fd-b84750cd9c1c

[26] Ross Freeman, James V. Barnett, and Bernard V. Vonderschmitt. 2021. Xilinx Edge Processors. In *2021 IEEE Hot Chips 33 Symposium (HCS)* (2021-08). IEEE, Palo Alto, CA, USA, 1–21. doi:10.1109/HCS52781.2021.9567521

[27] Lovic Gauthier and Yohei Ishikawa. 2024. HDLRuby: A Ruby Extension for Hardware Description and Its Translation to Synthesizable Verilog HDL. *ACM Trans. Embed. Comput. Syst.* 23, 5 (Aug. 2024), 73:1–73:26. doi:10.1145/3581757

[28] Érdi Gergő. 2021. *Retrocomputing with Clash: Haskell for FPGA Hardware Design*. LeanPub, Online. https://unsafeperform.io/retroclash/

[29] Dan R. Ghica. 2007. Geometry of Synthesis: A Structured Approach to VLSI Design. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. Association for Computing Machinery, 363–375. doi:10.1145/1190216.1190269

[30] Dan R. Ghica and Achim Jung. 2016. Categorical Semantics of Digital Circuits. In *2016 Formal Methods in Computer-Aided Design (FMCAD)*. 41–48. doi:10.1109/FMCAD.2016.7886659

[31] Thorsten Grotker, Stan Liao, Grant Martin, and Stuart Swan. 2010. *System Design with SystemC (TM)*. Springer, New York, NY.

[32] Xiaolong Guo, Raj Gautam Dutta, Jiaji He, Mark M. Tehranipoor, and Yier Jin. 2019. QIF-Verilog: Quantitative Information-Flow based Hardware Description Languages for Pre-Silicon Security Assessment. In *2019 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, McLean, VA, USA, 91–100. doi:10.1109/HST.2019.8740840

[33] N. Halbwachs and F. Maraninchi. 1995. On the Symbolic Analysis of Combinational Loops in Circuits and Synchronous Programs. In *Euromicro*. IEEE, Como (Italy), 1–7.

[34] Sungsoo Han, Minseong Jang, and Jeehoon Kang. 2023. ShakeFlow: Functional Hardware Description with Latency-Insensitive Interface Combinators. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 702–717. ISBN 978-1-4503-9916-6. doi:10.1145/3575693.3575701

[35] Naoya Hatta, Taichi Ishitani, and Ryota Shioya. 2024. Veryl: A New Hardware Description Language as an Altarnative to SystemVerilog. arXiv:2411.12983 [cs.AR]

[36] Pavol Hell and Jaroslav Nesetril. 2004. *Graphs and Homomorphisms*. Oxford University Press. ISBN 9780198528173. doi:10.1093/acprof:oso/9780198528173.001.0001

[37] C. A. R. Tony Hoare, Bernhard Möller, Georg Struth, and Ian Wehrman. 2009. Concurrent Kleene Algebra. In *CONCUR 2009 - Concurrency Theory*, Mario Bravetti and Gianluigi Zavattaro (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 399–414. ISBN 978-3-642-04081-8.

[38] Shan Shan Huang, Amir Hormati, David F. Bacon, and Rodric Rabbah. 2008. Liquid Metal: Object-Oriented Programming Across the Hardware/Software Boundary. In *ECOOP 2008 – Object-Oriented Programming*, Jan Vitek (Ed.). Springer, Berlin, Heidelberg, 76–103. ISBN 978-3-540-70592-5. doi:10.1007/978-3-540-70592-5_5

[39] IEEE. 2006. IEEE Standard for Verilog Hardware Description Language. 590 pages. doi:10.1109/IEEESTD.2006.99495

[40] IEEE. 2019. IEEE Standard for VHDL Language Reference Manual. 673 pages. doi:10.1109/IEEESTD.2019.8938196

[41] IEEE. 2024. IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language. doi:10.1109/IEEESTD.2024.10458102

[42] James Jennings and Eric Beuscher. 2000. Verischemelog: Verilog Embedded in Scheme. In *Proceedings of the 2nd Conference on Domain-specific Languages (DSL '99)*. Association for Computing Machinery, New York, NY, USA, 123–134. ISBN 978-1-58113-255-7. doi:10.1145/331960.331978

[43] David Karchmer and Daniel S. Stellenberg. 2004. Using Assignment Decision Diagrams with Control Nodes for Sequential Review during Behavioral Simulation. https://patents.google.com/patent/US6697773B1/en

[44] George Kaye. 2025. Foundations of Digital Circuits: Denotation, Operational, and Algebraic Semantics. arXiv:2502.08497 [cs] doi:10.48550/arXiv.2502.08497

[45] David Koeplinger, Matthew Feldman, Raghu Prabhakar, Yaqi Zhang, Stefan Hadjis, Ruben Fiszel, Tian Zhao, Luigi Nardi, Ardavan Pedram, Christos Kozyrakis, and Kunle Olukotun. 2018. Spatial: A Language and Compiler for Application Accelerators. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*. Association for Computing Machinery, New York, NY, USA, 296–311. ISBN 978-1-4503-5698-5. doi:10.1145/3192366.3192379

[46] Anish Krishnakumar, Umit Ogras, Radu Marculescu, Mike Kishinevsky, and Trevor Mudge. 2023. Domain-Specific Architectures: Research Problems and Promising Approaches. *ACM Transactions on Embedded Computing Systems* 22, 2 (Jan. 2023), 28:1–28:26. doi:10.1145/3563946

[47] Martin Kristien, Bruno Bodin, Michel Steuwer, and Christophe Dubach. 2019. High-Level Synthesis of Functional Patterns with Lift. In *Proceedings of the 6th ACM SIGPLAN International Workshop on Libraries, Languages and Compilers for Array Programming*. ACM, Phoenix AZ USA, 35–45. ISBN 978-1-4503-6717-2. doi:10.1145/3315454.3329957

[48] Johannes A. La Poutré. 1994. Alpha-algorithms for incremental planarity testing (preliminary version). In *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing* (Montreal, Quebec, Canada) *(STOC '94)*. Association for Computing Machinery, New York, NY, USA, 706–715. ISBN 0897916638. doi:10.1145/195058.195439

[49] Bluespec Lang. 2026. Bluespec Compiler. https://github.com/B-Lang-org/bsc

[50] Tony Law, Delphine Demange, and Sandrine Blazy. 2025. A Mechanized Semantics for Dataflow Circuits. *Proc. ACM Program. Lang.* 9 (2025), 98:507–98:533. Issue OOPSLA1. doi:10.1145/3720432

[51] Xun Li, Mohit Tiwari, Jason K. Oberg, Vineeth Kashyap, Frederic T. Chong, Timothy Sherwood, and Ben Hardekopf. 2011. Caisson: A Hardware Description Language for Secure Information Flow. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 109–120. ISBN 978-1-4503-0663-8. doi:10.1145/1993498.1993512

[52] Yamin Li and Wanming Chu. 1996. A New Non-Restoring Square Root Algorithm and Its VLSI Implementations. In *Proceedings International Conference on Computer Design. VLSI in Computers and Processors*. IEEE, Austin, TX, USA, 538–544. doi:10.1109/ICCD.1996.563604

[53] Richard Lin, Rohit Ramesh, Connie Chi, Nikhil Jain, Ryan Nuqui, Prabal Dutta, and Björn Hartmann. 2020. Polymorphic Blocks: Unifying High-level Specification and Low-level Control for Circuit Board Design. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology (UIST '20)*. Association for Computing Machinery, New York, NY, USA, 529–540. ISBN 978-1-4503-7514-6. doi:10.1145/3379337.3415860

[54] Richard Lin, Rohit Ramesh, Antonio Iannopollo, Alberto Sangiovanni Vincentelli, Prabal Dutta, Elad Alon, and Björn Hartmann. 2019. Beyond Schematic Capture: Meaningful Abstractions for Better Electronics Design Tools. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*. Association for Computing Machinery, New York, NY, USA, 1–13. ISBN 978-1-4503-5970-2. doi:10.1145/3290605.3300513

[55] Andreas Lööw. 2022. A Small, but Important, Concurrency Problem in Verilog's Semantics? (Work in Progress). In *2022 20th ACM-IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE)*. IEEE, "Shanghai, China", 1–6. doi:10.1109/MEMOCODE57689.2022.9954591

[56] Andreas Lööw. 2025. The Simulation Semantics of Synthesisable Verilog. *Proc. ACM Program. Lang.* 9 (2025), 126:1295–126:1320. Issue OOPSLA1. doi:10.1145/3720484

[57] K. Maharatna, E. Grass, and U. Jagdhold. 2004. A 64-Point Fourier Transform Chip for High-Speed Wireless LAN Application Using OFDM. *IEEE Journal of Solid-State Circuits* 39, 3 (2004), 484–493. doi:10.1109/JSSC.2003.822776

[58] S. Malik. 1994. Analysis of Cyclic Combinational Circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 13, 7 (1994), 950–956. doi:10.1109/43.293952

[59] David A. Mellis, Leah Buechley, Mitchel Resnick, and Björn Hartmann. 2016. Engaging Amateurs in the Design, Fabrication, and Assembly of Electronic Devices. In *Proceedings of the 2016 ACM Conference on Designing Interactive Systems (DIS '16)*. Association for Computing Machinery, New York, NY, USA, 1270–1281. ISBN 978-1-4503-4031-1. doi:10.1145/2901790.2901833

[60] Robin Milner. 1993. The Polyadic π-Calculus: a Tutorial. In *Logic and Algebra of Specification*, Friedrich L. Bauer, Wilfried Brauer, and Helmut Schwichtenberg (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 203–246. ISBN 978-3-642-58041-3.

[61] Rachit Nigam, Pedro Henrique Azevedo de Amorim, and Adrian Sampson. 2023. Modular Hardware Design with Timeline Types. *Proc. ACM Program. Lang.* 7 (2023), 120:343–120:367. Issue PLDI. doi:10.1145/3591234

[62] Rachit Nigam, Ethan Gabizon, Edmund Lam, Carolyn Zech, Jonathan Balkind, and Adrian Sampson. 2026. Parameterized Hardware Design with Latency-Abstract Interfaces. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (USA) *(ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1382–1395. ISBN 9798400723599. doi:10.1145/3779212.3790199

[63] R. Nikhil. 2004. Bluespec System Verilog: Efficient, Correct RTL from High Level Specifications. In *Proceedings. Second ACM and IEEE International Conference on Formal Methods and Models for Co-Design, 2004. MEMOCODE '04.* IEEE, San Diego, CA, USA, 69–70. doi:10.1109/MEMCOD.2004.1459818

[64] David A. Patterson and John L. Hennessy. 2017. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface* (1st ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA. ISBN 0128122757.

[65] Benjamin C. Pierce (Ed.). 2005. *Advanced Topics in Types and Programming Languages.* MIT Press, Cambridge, Mass. ISBN 978-0-262-16228-9.

[66] Oron Port and Yoav Etsion. 2017. DFiant: A Dataflow Hardware Description Language. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, Ghent, Belgium, 1–4. ISBN 978-90-90-30428-1. doi:10.23919/FPL.2017.8056858

[67] Iain E. Richardson. 2010. *The H.264 Advanced Video Compression Standard* (2nd ed.). Wiley Publishing. ISBN 0470516925.

[68] Cherif Salama, Gregory Malecha, Walid Taha, Jim Grundy, and John O'Leary. 2009. Static Consistency Checking for Verilog Wire Interconnects: Using Dependent Types to Check the Sanity of Verilog Descriptions. In *Proceedings of the 2009 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation (PEPM '09)*. Association for Computing Machinery, New York, NY, USA, 121–130. ISBN 978-1-60558-327-3. doi:10.1145/1480945.1480963

[69] Martine Schlag. 1987. The Planar Topology of Functional Programs. In *Functional Programming Languages and Computer Architecture*, Gilles Kahn (Ed.). Springer, Berlin, Heidelberg, 174–193. ISBN 978-3-540-47879-9. doi:10.1007/3-540-18317-5_11

[70] Martin Schoeberl. 2019. Digital Design with Chisel. https://github.com/schoeberl/chisel-book

[71] Fabian Schuiki, Andreas Kurth, Tobias Grosser, and Luca Benini. 2020. LLHD: A Multi-Level Intermediate Representation for Hardware Description Languages. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 258–271. ISBN 978-1-4503-7613-6. doi:10.1145/3385412.3386024

[72] S. Singh and P. James-Roxby. 2001. Lava and JBits: From HDL to Bitstream in Seconds. In *The 9th Annual IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM'01)*. IEEE, USA, 91–100.

[73] Zachary D. Sisco, Sijie Kong, Daniel Ruelas-Petrisko, Jingtao Xia, Julian Springer, Varun Rao, Spencer Wang, Gus Henry Smith, Ben Hardekopf, and Jonathan Balkind. 2026. Fungible Memories for Automated Technology Mapping and Retargeting. *Proceedings of the ACM on Programming Languages* 10, PLDI (June 2026), 1178–1201. doi:10.1145/3808294

[74] Frans Skarman, Gustav Sörnäs, and Oscar Gustafsson. 2026. Spade—A Modern Hardware Description Language. *ACM Transactions on Reconfigurable Technology and Systems* 19, 2, Article 21 (2026), 28 pages. doi:10.1145/3793550

[75] Wilson Snyder. 2025. Verilator. https://www.veripool.org/wiki/verilator

[76] RISC-V Software Source. 2025. RISC-V Tests. https://github.com/riscv-software-src/riscv-tests

[77] Stuart Sutherland, Don Mills, and Chris Spear. 2007. Gotcha Again: More Subtleties in the Verilog and SystemVerilog Standards That Every Engineer Should Know. *SNUG'07 (Synopsys Users Group San Jose, CA, 1998) Proceedings* (2007).

[78] Omar Tahir, Wayne Luk, and Nicolas Wu. 2023. Extensible Embedded Hardware Description Languages with Compilation, Simulation and Verification. In *Proceedings of the 13th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies (HEART '23)*. Association for Computing Machinery, New York, NY, USA, 1–10. ISBN 979-8-4007-0043-9. doi:10.1145/3597031.3597051

[79] The Rocq Development Team. 2025. *The Rocq Prover.* doi:10.5281/zenodo.15149628

[80] Donald Thomas and Philip Moorby. 2008. *The Verilog® Hardware Description Language.* Springer Science & Business Media, New York, Boston, Dordrecht, London, Moscow. ISBN 978-0-387-85344-4.

[81] Yongding Tian, Matthijs Reukers, Zaid Al-Ars, Peter Hofstee, Matthijs Brobbel, Johan Peltenburg, and Jeroen Straten. 2023. Tydi-Lang: A Language for Typed Streaming Hardware. In *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W '23)*. Association for Computing Machinery, New York, NY, USA, 521–529. ISBN 9798400707858. doi:10.1145/3624062.3624539

[82] Richard Townsend, Martha A. Kim, and Stephen A. Edwards. 2017. From Functional Programs to Pipelined Dataflow Circuits. In *Proceedings of the 26th International Conference on Compiler Construction (CC 2017)*. Association for Computing Machinery, New York, NY, USA, 76–86. ISBN 978-1-4503-5233-8. doi:10.1145/3033019.3033027

[83] Megan Wachs. 2022. Clarifying Directionality in Chisel Types #2643. https://github.com/chipsalliance/chisel/issues/2643

[84] Francis Wang. 2022. *Verik: Reinterpreting Kotlin as a Hardware Description Language.* Thesis. Massachusetts Institute of Technology.

[85] Yager Will. 2015. HaSKI: An FPGA-based SKI calculus evaluator written in Cλash/Haskell. https://github.com/wyager/HaSKI

[86] Stephen Williams. 2025. Icarus Verilog. https://github.com/steveicarus/iverilog

[87] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2024. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg. ISBN 978-3-662-69305-6 978-3-662-69306-3. doi:10.1007/978-3-662-69306-3

[88] A. K. Wright and M. Felleisen. 1994. A Syntactic Approach to Type Soundness. *Information and Computation* 115, 1 (1994), 38–94. doi:10.1006/inco.1994.1093

[89] Yinan Xu, Zihao Yu, Dan Tang, Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, Zuojun Li, Jiawei Lin, Tong Liu, Zhigang Liu, Jiazhan Tan, Huaqiang Wang, Huizhe Wang, Kaifan Wang, Chuanqi Zhang, Fawang Zhang, Linjuan Zhang, Zifei Zhang, Yangyang Zhao, Yaoyang Zhou, Yike Zhou, Jiangrui Zou, Ye Cai, Dandan Huan, Zusong Li, Jiye Zhao, Zihao Chen, Wei He, Qiyuan Quan, Xingwu Liu, Sa Wang, Kan Shi, Ninghui Sun, and Yungang Bao. 2022. Towards Developing High Performance RISC-V Processors Using Agile Methodology. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)* (2022-10). IEEE, Chicago, IL, USA, 1178–1199. doi:10.1109/MICRO56248.2022.00080

[90] Yanbing Li and M. Leeser. 2000. HML, a Novel Hardware Description Language and Its Translation to VHDL. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 8, 1 (Feb. 2000), 1–8. doi:10.1109/92.820756

[91] Danfeng Zhang, Yao Wang, G. Edward Suh, and Andrew C. Myers. 2015. A Hardware Design Language for Timing-Sensitive Information-Flow Security. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, Istanbul Turkey, 503–516. ISBN 978-1-4503-2835-7. doi:10.1145/2694344.2694372

A Supplementary Material

A.1 Extension to Formalism

In the main text, we omitted some details and proofs of the formalism of λ_{NX} due to space constraints. Here, we provide the additional details and proofs for interested readers.

A.1.1 Elaboration Rules. Elaboration is the process of eliminating meta-programming constructs to obtain a normal form that preserves the schematic core only. As specified in the main text, the normal form is defined as follows:

$$\bar{e} ::= x \mid \bar{e} \diamond \bar{e} \mid (\bar{e} \mid\mid \bar{e}) \mid \bar{e}\# \mid \#\bar{e} \mid n \mid \lambda x.\, e$$

The elaboration rules follow standard beta-reduction in λ-calculus. In particular, we adopt a call-by-value small-step semantics to formalize the elaboration $e_1 \hookrightarrow e_2$, as shown in Fig. 15. Here, the notations include: (1) E is an elaboration context [22, 88] and $E[e]$ is the expression obtained by substituting the $[\cdot]$ in E with e; (2) $e_1[x \mapsto e_2]$ represents the expression obtained by substituting x with e_2 in e_1; and (3) $[\![\text{op}]\!]$ denotes the semantic function of the operator op.

$$E ::= [\cdot] \mid E \diamond e \mid \bar{e} \diamond E \mid (E \mid\mid e) \mid (\bar{e} \mid\mid E) \mid E\# \mid \#E \mid E(e) \mid \bar{e}(E) \mid \texttt{if}\ E\ \texttt{then}\ e\ \texttt{else}\ e \mid E \text{ op } e \mid \bar{e} \text{ op } E$$

$$\frac{e \hookrightarrow e'}{E[e] \hookrightarrow E[e']}\ \text{E-Ctx}$$

$$\frac{}{(\lambda x : \tau.\, e_1)(\bar{e}_2) \hookrightarrow e_1[x \mapsto \bar{e}_2]}\ \text{E-Call} \qquad \frac{}{\texttt{let}\ x : \tau = e_1\ \texttt{in}\ e_2 \hookrightarrow e_2[x \mapsto e_1[x \mapsto \texttt{let}\ x : \tau = e_1\ \texttt{in}\ x]]}\ \text{E-Let}$$

$$\frac{\bar{e} \neq 0}{\texttt{if}\ \bar{e}\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \hookrightarrow e_2}\ \text{E-Cond-T} \qquad \frac{\bar{e} = 0}{\texttt{if}\ \bar{e}\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \hookrightarrow e_3}\ \text{E-Cond-F} \qquad \frac{[\![\text{op}]\!] = \delta \qquad \delta(\bar{e}_1, \bar{e}_2) = n}{\bar{e}_1 \text{ op } \bar{e}_2 \hookrightarrow n}\ \text{E-Op}$$

Fig. 15. Elaboration Rules for λ_{NX}.

For example, E-Call performs β-reduction by substituting the argument into the function body, while E-Let unfolds a let-binding by substituting the function body into itself, enabling recursion. Note the distinction between $\bar{e}$ and e in the rules: $\bar{e}$ denotes the normal form, and its appearance in the rules indicates the elaboration order. Elaboration is deterministic: each expression has at most one next elaboration step and, if elaboration terminates, it reaches a unique normal form.

Theorem A.1 (E-Determinism). $\forall e, e_1, e_2.\ (e \hookrightarrow e_1 \wedge e \hookrightarrow e_2) \Rightarrow (e_1 \equiv e_2)$.

Proof Sketch. First prove that normal forms cannot take an elaboration step simply by induction on the structure of $\bar{e}$. Then, we proceed by structural induction on e, inverting the two elaboration derivations in each case. Since the call-by-value elaboration contexts are disjoint, the two derivations must reduce the same subexpression, and the induction hypothesis gives identical results; thus, the two derivations must produce the same result. □

A.1.2 Type Soundness. Type soundness is a standard property of programming languages and can be reinterpreted as a consistency-related property in λ_{NX}, in contrast to various problems in behavioral HDLs discussed in the main text.

Here, we state the preservation and progress properties that establish type soundness for λ_{NX}, following standard formulations [65]: elaboration preserves types, and well-typed elaborated circuit expressions generate graphs with matching IO interfaces.

(a) Starting from the type uniqueness property (an expression can only have one unique type in a given context) that serves as a foundation for the type system, we have:

Theorem A.2 (T-Uniqueness). $\forall \Gamma, e, \tau_1, \tau_2.\ (\Gamma \vdash e : \tau_1 \wedge \Gamma \vdash e : \tau_2) \Rightarrow (\tau_1 = \tau_2)$.

Proof Sketch. We proceed by structural induction on e, inverting the two typing derivations in each case. For identifiers, both derivations use the same context lookup and therefore give the same type. For compound expressions, the induction hypotheses identify the types of their subexpressions, and the corresponding typing rule uniquely determines the result type. □

(b) Then, we can prove the preservation theorems stating that the type of an expression is preserved between each step of elaboration and evaluation (steps do not break typing), respectively:

Theorem A.3 (E-Preservation). $\forall \Gamma, e, \tau, e'.\ (e \hookrightarrow e') \wedge (\Gamma \vdash e : \tau) \Rightarrow (\Gamma \vdash e' : \tau)$.

Proof Sketch. First prove the substitution lemma: substituting a well-typed expression for a meta-variable of the same type preserves typing. Then, we proceed by structural induction on e, inverting its elaboration and typing derivations. Contextual steps follow from the induction hypothesis; function application and recursive let-unfolding use the substitution lemma, while conditional and arithmetic reductions preserve the result type directly. □

Theorem A.4 (G-Preservation). $\forall \Gamma, \bar{e}, i, o, \mathrm{G}.\ (\Gamma, \bar{e} \Rightarrow \mathrm{G}) \wedge (\Gamma \vdash \bar{e} : i \rightsquigarrow o) \Rightarrow (\mathrm{G} : i \rightsquigarrow o)$.

Proof Sketch. We proceed by structural induction on $\bar{e}$, inverting its evaluation and typing derivations. For primitive components, type uniqueness identifies the declared interface, and vertex construction preserves its signal types. For composition, the induction hypotheses give the interfaces of the constituent graphs; index shifting preserves signal types, while parallel composition concatenates the interfaces. The blocking rules remove the corresponding interface, as required by their typing rules. □

Here, $\mathrm{G} : i \rightsquigarrow o$ denotes that in $\mathrm{G} = (\mathrm{V}, \mathrm{E}, \mathrm{I}, \mathrm{O})$, extracting the type annotations in I and O gives $i \rightsquigarrow o$, e.g., from $\mathrm{I} = \langle 1 : \mathtt{b1}, 2 : \mathtt{clock} \rangle$ and $\mathrm{O} = \langle 3 : \mathtt{b2} \rangle$ we have $\mathrm{G} : \langle \mathtt{b1}, \mathtt{clock} \rangle \rightsquigarrow \langle \mathtt{b2} \rangle$.

(c) Further, we have the progress theorems stating that a well-typed expression is either a value or can take a step of elaboration or evaluation, respectively.

Theorem A.5 (E-Progress). $\forall \Gamma, e, \tau.\ (\Gamma \vdash e : \tau) \Rightarrow (\exists \bar{e}.\ e = \bar{e}) \vee (\exists e'.\ e \hookrightarrow e')$.

Proof Sketch. We proceed by structural induction on e, inverting its typing derivation in each case. For compound expressions, the induction hypotheses either provide a step in the first non-normal operand or establish that the operands are normal forms. In the latter case, circuit constructs are already normal, while typing ensures that applications, conditionals, and arithmetic expressions have operands of the required forms and can reduce. Recursive let-bindings take an unfolding step directly. □

Theorem A.6 (G-Progress). $\forall \Gamma, \bar{e}, i, o.\ (\Gamma \vdash \bar{e} : i \rightsquigarrow o) \Rightarrow (\exists \mathrm{G}.\ \Gamma, \bar{e} \Rightarrow \mathrm{G})$.

Proof Sketch. We proceed by structural induction on the elaborated circuit expression $\bar{e}$. Primitive components generate graphs directly, and the induction hypotheses provide graphs for the operands of composition and blocking. For serial composition, G-Preservation ensures that the connected interfaces have matching signal types, so the connecting wires can be constructed. Parallel composition and blocking then construct their graphs directly. □

A.1.3 Simulation Semantics. The main text presents the semantics for constructing circuit graphs from λ_{NX} programs. Here, we describe how to interpret these graphs as reactive systems to simulate circuit behavior, specifying how a circuit reacts to external inputs and produces responses.

To define the behavior of the circuit graph, below we provide the simulation semantics of λ_{NX} that interprets the circuit graph as an intuitive *reactive system*. In the system, each vertex carries a signal value; reactive means once the value of a vertex changes, it will trigger the update of other vertices along the hyperedges (components) accordingly. That is, the system reacts to external stimuli and produces responses by propagating signal changes through the circuit graph.

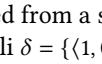
Formally, first we make a convention on the possible labels ℓ of the hyperedges (i.e., the kinds of components) without loss of generality. We adopt the following components that are known to be sufficient to represent any circuit [5]: (a) named wires x_i and anonymous wires α; (b) NAND : $\langle \mathsf{b1}, \mathsf{b1} \rangle \rightsquigarrow \langle \mathsf{b1} \rangle$ that outputs 0 only if both its inputs are 1; and (c) REGISTER : $\langle \mathsf{clock}, \mathsf{b1} \rangle \rightsquigarrow \langle \mathsf{b1} \rangle$ that stores its data input when its clock input rises from 0 to 1, outputting the stored value.

Given the hypergraph $\mathsf{G} = (\mathsf{V}, \mathsf{E}, \mathsf{I}, \mathsf{O})$, we define the simulation transducer as a pair (σ, δ), where $\sigma : \mathbb{N} \to \{0, 1\}$ represents the current state of all vertices, and $\delta \subset \mathbb{N} \times \{0, 1\}$ is the set of pending update events that should happen simultaneously in the next transition step. For example, $\sigma(u) = b$ means that vertex u has value b, and $\langle u, b \rangle \in \delta$ means that vertex u should be set to value b in the next step. The simulation transition for the circuit graph G is written as: $(\sigma, \delta) \rightsquigarrow_{\mathsf{G}} (\sigma', \delta')$, where:

(1) We compute the new state $\sigma' = \sigma[u \mapsto b \mid \langle u, b \rangle \in \mathbf{fix}\ \kappa(\delta)]$. First, consider the semantics of wires (x_i and α): when there is an update event on one end of a wire, it is propagated to the other end. Denote the closure of all update events after propagation as $\mathbf{fix}\ \kappa(\delta)$, then we apply these events simultaneously to σ and get the new state σ'.
Here, κ denotes the single-step propagation over wires expanding δ. Specifically, for all $\langle u, b \rangle \in \delta$: (a) if there is a hyperedge $v \overset{\alpha}{\leftarrow} u$ or $v \overset{x_i}{\leftarrow} u$, then $\langle u, b \rangle \in \delta$ implies $\langle v, b \rangle \in \kappa(\delta)$. (b) moreover, if there are two instances of the same named wire $u' \overset{x_i}{\leftarrow} u$ and $v' \overset{x_i}{\leftarrow} v$, since wires of the same name should carry the same signal, $\langle u, b \rangle \in \delta$ implies $\langle v, b \rangle \in \kappa(\delta)$. Formally, define:
$$\kappa(\delta) = \delta \cup \{\langle v, b \rangle \mid \forall \langle u, b \rangle \in \delta.\ (\mathsf{E}(u, v) = \alpha) \vee (\mathsf{E}(u, v) = x_i) \vee (\mathsf{E}(u, u') = x_i \wedge \mathsf{E}(v, v') = x_i)\}$$
Note that an update event propagated through a wire may be further propagated through other wires. Therefore, we keep applying κ until no new update events can be further propagated; that is, formally, we compute the fixed point of κ, denoted as $\mathbf{fix}\ \kappa(\delta)$.

(2) Then, we compute the new set of update events $\delta' = \bigcup_{\mathsf{E}(i,o)=\ell} [\![\ell]\!]\ (i, o)$. Since the current state has been updated, there may be some components that need to recompute their outputs according to their new inputs. Using $[\![\ell]\!]$ to denote the semantics of the component ℓ, for each component (i.e., for each hyperedge $\mathsf{E}(i, o) = \ell$, where i, o are its inputs and outputs and ℓ is its label), we compute the new update events it triggers as $[\![\ell]\!]\ (i, o)$. By taking the union of all such events, finally we get δ'. Specifically, below we define the semantics of NAND and REGISTER:
(a) NAND outputs 0 only if both its inputs are 1. If the recomputed $\neg(\sigma'(u_1) \wedge \sigma'(u_2))$ mismatches the output $\sigma(v)$, it triggers an update event to flip its output; otherwise, it triggers no events.
$$[\![\mathsf{NAND}]\!] = (\langle u_1, u_2 \rangle, v) \mapsto \begin{cases} \{\langle v, \neg\sigma(v) \rangle\} & \sigma(v) \neq \neg(\sigma'(u_1) \wedge \sigma'(u_2)) \\ \emptyset & \text{otherwise.} \end{cases}$$
(b) REGISTER outputs the value of its data input (u) at the last *rising edge*, where a rising edge is the transition of its clock input (clk) from 0 to 1. If the output differs from this value, it triggers an update event.
$$[\![\mathsf{REGISTER}]\!] = (\langle \mathsf{clk}, u \rangle, v) \mapsto \begin{cases} \{\langle v, \neg\sigma(v) \rangle\} & \sigma(\mathsf{clk}) = 0, \sigma'(\mathsf{clk}) = 1, \sigma(v) \neq \sigma'(u) \\ \emptyset & \text{otherwise.} \end{cases}$$

During simulation, we can send stimuli to the circuit by modifying δ. Note that it may take multiple steps to propagate the stimuli and reach $\delta = \emptyset$, and only then can we send new stimuli again.

Example A.7. Consider the graph evaluated from a single NAND, i.e., 

. Given the initial state $\sigma = \{1 \mapsto 1, 2 \mapsto 1, 3 \mapsto 0\}$ and the stimuli $\delta = \{\langle 1, 0 \rangle\}$, we have:
$$(\sigma, \delta) \rightsquigarrow_{\mathsf{G}} (\{1 \mapsto 0, 2 \mapsto 1, 3 \mapsto 0\}, \{\langle 3, 1 \rangle\}) \rightsquigarrow_{\mathsf{G}} (\{1 \mapsto 0, 2 \mapsto 1, 3 \mapsto 1\}, \emptyset)$$
Here the delta of each step is highlighted in red. First, we apply $\delta = \{\langle 1, 0 \rangle\}$ to σ, updating vertex 1's value to 0. Then, according to $[\![\mathsf{NAND}]\!]$, since the inputs are now 0 and 1, the output 3 should change

to 1, thus we have a new pending update event $\langle 3, 1\rangle$. Finally, we apply this new event to σ, resulting in the final state. Also, since there are no more pending events ($\delta = \emptyset$), we can send new stimuli again.

Remark A.8 (Comparison with Other Semantics). The reactive-system style of λ_{NX}'s simulation semantics differs from other languages in that λ_{NX} does not involve sophisticated scheduling queues of various kinds of events (e.g., blocking, non-blocking, continuous, and delayed events) and their interactions, which are known to be a source of subtle and counter-intuitive pitfalls [13, 56].

A.1.4 Proofs of Theorems in the Main Text. The following sketches explain the proofs of the three theorems and the acyclicity corollary presented in the main text.

Theorem 4.5 (G-Ascending). $\forall \Gamma, e, \mathrm{G}.\ (\Gamma, e \Rightarrow \mathrm{G}) \Rightarrow (\forall \langle i, o\rangle \in \mathrm{dom}(\mathsf{E}).\ \forall u \in i.\ \forall v \in o.\ u < v)$.

Proof Sketch. We proceed by structural induction on the source expression, inverting its graph-evaluation derivation. Primitive components assign smaller indices to inputs than to outputs. Index shifting preserves this ordering, and in serial composition, every vertex of G_2 has an index below $|\mathsf{V}_2|$, while every vertex of the shifted G_1 has an index at least $|\mathsf{V}_2|$, so each connecting wire also points from a smaller index to a larger one. Parallel composition combines disjoint graphs and introduces no edges, so it would not break the property; similarly the blocking rules leave the edges unchanged and preserve the property; thus, every generated hyperedge preserves the strict ordering from vertices with smaller indices to the larger ones. □

Corollary 4.6. *The graph* G *is acyclic since its vertex indices induce a valid topological order.*

Proof Sketch. By G-Ascending, each step along a directed path strictly increases the vertex index. A directed cycle would therefore require its starting vertex to have a strictly larger index than itself, a contradiction. This argument concerns the generated graph before identifying occurrences of the same named wire. □

Theorem 4.7 (G-Planarity). $\forall \Gamma, e, \mathrm{G}.\ (\Gamma, e \Rightarrow \mathrm{G}) \Rightarrow \mathrm{G}$ *is planar, i.e., its vertices and hyperedges admit an embedding in the Euclidean plane such that no two connecting segments intersect.*

Proof Sketch. We prove planarity by assigning each vertex and hyperedge a distinct coordinate $(x, y) \in \mathbb{N}^2$ and drawing a line segment for each incidence between them. We show that these segments meet only at shared endpoints and never pass through other points. For example, •←AND⊐ becomes •-<, where the middle point corresponds to the hyperedge and the other three points correspond to the vertices, with no segments intersecting.

To compose these drawings without crossings, we also keep output ports on the left boundary and input ports on the right boundary, with each list of ports ordered vertically. We prove by induction that each evaluation rule produces a drawing with these properties:

- *G-Comp.* A primitive component becomes a star-shaped incidence graph, with a central point for its hyperedge and separate points for its input and output vertices. For a signal vertex with index j, assign coordinate $(2, j + 1)$ if it is an input and $(0, j + 1)$ if it is an output, and place the component's hyperedge point at $(1, 1)$. The distinct port coordinates and opposite boundaries give a noncrossing star embedding with ordered interfaces.
- *G-Par.* For the parallel composition $\mathrm{G}_1 \mid\mid \mathrm{G}_2$, the index shifting makes their vertex sets disjoint without changing their topology. First rescale the horizontal coordinates so that both embeddings for G_1 and G_2 have the same width W. Keep the coordinates in the G_1 embedding unchanged, and map each point (x, y) of the G_2 embedding to $(x, y + H)$, where H exceeds every vertical coordinate in the G_1 embedding. This separates the embeddings vertically while preserving the concatenated interface order. Since neither operand adds connections to the other, this yields a noncrossing embedding of the parallel composition and thus preserves the property.

- *G-Seq.* For the serial composition $G_1 <> G_2$, first rescale both embeddings to width W, keep G_1 on the left, and map each point (x, y) of G_2 to $(x + W + 2, y)$ on the right. Thus, the input boundary of G_1 lies at $x = W$, facing the output boundary of G_2 at $x = W + 2$. Place the incidence-graph point for the k-th anonymous connecting wire at $(W + 1, k + 1)$, and connect it to the corresponding ports on the two boundaries. These connecting paths can be drawn without intersecting one another or either operand's interior, and the unconnected outer interfaces remain ordered.
- *G-Blk-I and G-Blk-O.* Blocking removes an exposed interface but leaves all vertices and hyperedges unchanged. Keep every point at its existing coordinate; the embedding remains valid, and the ordering requirement for the hidden interface becomes vacuous.

Finally, every evaluation rule preserves a drawing in which distinct points have distinct coordinates and segments meet only at shared endpoints. The resulting circuit graph therefore is guaranteed to be planar. □

Theorem 4.9 (Local-Correspondence). *For a whole execution $e \hookrightarrow^* \bar{e} \Rightarrow G$, each subexpression of e yields $n \in \mathbb{N}$ subexpressions $\bar{e}_1, \ldots, \bar{e}_n$ of $\bar{e}$ and for each $\bar{e}_i \Rightarrow G_i$, G_i is a subgraph of* G.

Proof Sketch. The proof follows three steps along the execution $e \hookrightarrow^* \bar{e} \Rightarrow G$:

- *Tracking instances through elaboration.* The main difficulty is that meta-programming may duplicate, instantiate, or eliminate a source AST subtree, so it need not correspond to a single subtree after elaboration. We mark the chosen source occurrence and propagate its provenance through substitution, function application, and recursive unfolding, retaining multiplicities when instances are copied or merged. Every ordinary elaboration step lifts to a marked step whose erasure gives the same expression; induction over the finite elaboration sequence therefore yields a finite list of resulting instances $\bar{e}_1, \ldots, \bar{e}_n$, possibly empty.
- *Locating instances in the normal form.* We prove that each resulting expression $\bar{e}_i$ occupies a subtree of $\bar{e}$ by structural induction on the normal form. If $\bar{e}_i$ occupies the root, the claim is immediate. Otherwise, it occurs within an operand of serial or parallel composition, or within the expression enclosed by a blocking operator #. The induction hypothesis makes it a subtree of that operand, and hence a subtree of the whole expression. These cases exhaust the circuit syntax of the normal forms, so each instance $\bar{e}_i$ is a subtree of $\bar{e}$.
- *Embedding their circuit graphs.* Finally, we prove that each subtree of $\bar{e}$ generates a *index-shifted subgraph* of G. Here, G′ is an index-shifted subgraph of G if there exists an offset k such that the graph $G' \triangleright k$ is a subgraph of those of G, i.e., we focus on schematic shapes rather than the specific vertex indices when considering the subgraph relation. Then, we proceed by structural induction on the normal form $\bar{e}$: the hypothesis gives the subgraph property for each operand of the operators (<>, ||, and #), and the evaluation rules for these operators construct the overall graph by combining the subgraphs of their operands, preserving the subgraph property. □

A.2 Extension to Implementation and Evaluation

In the main text, we omit some details of the implementation and evaluation of the NetX toolchain due to space constraints, and we provide them here for completeness.

A.2.1 Benchmark Overview. Table 3 summarizes the 12 benchmarks implemented in NetX, spanning application domains, complexity levels, and hardware design patterns. The benchmarks are selected from textbooks, papers, and projects [9, 52, 57, 64, 67], with two considerations:

(1) *Application diversity.* To demonstrate that NetX can express a range of practical hardware structures, we select benchmarks from multiple application domains. The suite covers signal processing (FFT), video compression (CAVLC), cryptography (AES-128 and SHA-256), networking

(ETHMAC), interconnects (AXI), and computer architecture (DDR2 and CPU). The smaller designs, LFSR, SQRT, SHIFT, and TIMER, exercise basic components used in larger systems.

(2) *Structural diversity.* To demonstrate that NetX can express a range of canonical design patterns that commonly appear in practical hardware, we select benchmarks with diverse structures. The designs exercise feedback loops (LFSR), iterative datapaths (SQRT), crossbar-style selection (SHIFT), and pipelined butterfly structures (FFT). They also include irregular wiring (CAVLC), finite state machines (TIMER and DDR2), complicated interconnects (AXI), and pipelined designs (CPU).

Although moderate in scale compared to industrial designs, the suite provides evidence that NetX can express a range of practical hardware structures, from small components to an integrated computer system. The table gives each benchmark's code size and functionality; the main-text case study examines the implementation, simulation, and debugging of the CPU system in more detail.

Table 3. Overview of the benchmarks implemented in NetX, showing the lines of NetX code (LoC), application domain, and a brief description of each design.

Project	LoC	Domain	Description
LFSR	31	Primitive	Galois linear-feedback shift register generating 16-bit pseudo-random numbers
SQRT	38	Primitive	Iterative component computing the square root of 32-bit integers in 16 cycles
SHIFT	39	Primitive	Barrel shifter supporting left/right, logical/arithmetic 32-bit shift operations
TIMER	55	Primitive	FPGA clock system displaying the current time on 7-segment outputs
FFT	132	DSP	8-point Fast Fourier Transform (FFT) on 48-bit fixed-point complex numbers, implemented with pipelined butterfly units and staged datapath composition
CAVLC	161	Video	Context-adaptive variable-length coding (CAVLC) engine used in H.264/AVC-style video compression, featuring irregular control and bit-level data manipulation
AES-128	227	Cryptography	Byte-streaming single-block Advanced Encryption Standard (AES) engine with 128-bit key input, round-based transformation logic, and ciphertext output
SHA-256	292	Cryptography	Byte-streaming single-block Secure Hash Algorithm (SHA) engine with internal padding, message-schedule expansion, and 256-bit digest output
ETHMAC	658	Networking	Buffered full-duplex Ethernet MAC with automatic padding, CRC32/FCS generation and checking, destination filtering, and MII-compatible nibble serialization
DDR2	776	Architecture	DDR2 memory controller with AXI4-Full interface, burst translation, open-row scheduling, per-bank timing enforcement, and refresh arbitration logic
AXI	912	Interconnect	AXI4-Full communication library supporting multiple devices, with burst-capable RAM, address decoding, transaction IDs, arbitration, and error handling
CPU	1914	Architecture	A computer system consisting of keyboard, monitor, and a 5-stage pipelined CPU with BTB-style branch predictor, following the RV32I instruction set architecture

A.2.2 Extension to Case Study. Example 4.3 and the case study in the main text suggest that NetX can facilitate optimizations of sequential circuits, such as pipelining and retiming.

First, we illustrate how to pipelining a circuit by a simple example from our FFT benchmark, a signal processing component that contains multiple butterfly units performing the computation of $a \times b - c \times d$.

```
let REG = REGISTER(pos_edge=true) <> [clk#, TO];
y <> REG <> SUB <> [REG, REG] <> [
    MUL <> [a, b], MUL <> [c, d]
];
```

The code snippet on the right shows a simplified butterfly datapath that performs the computation and stores the result in a register. When all four operands come from registers, the original datapath contains a register-to-register path through two components, a multiplier and a subtractor. This could be a performance bottleneck; therefore, to improve performance, we can pipeline the butterfly by splitting the long path into two shorter paths. We split this path by inserting the highlighted [REG, REG] between the subtractor and the multipliers, which is quite straightforward due to the compositional nature of NetX's schematic model. Now the cirtical path only contains a multiplier, which reduces the overall delay and therefore improves the maximum clock frequency; concretely,

this increases the maximum clock frequency from 108.60 MHz to 134.44 MHz, at the cost of using more registers and logic elements surrounding the new registers.

By contrast, to achieve a similar pipeline insertion in behavioral languages such as Verilog, the modification would typically involve creating additional intermediate signals and managing their wiring, for example as shown in the following code snippet:

```
always @(posedge clk)
  y <= a * b - c * d;
```

$\Longrightarrow$

```
reg [39:0] ab_r, cd_r;
always @(posedge clk) begin
  ab_r <= a * b;
  cd_r <= c * d;
  y    <= ab_r - cd_r;
end
```

Although both language can express the same pipelining optimization, NetX's schematic model allows the designer to express the optimization more concisely and can easily revert it if needed, while the Verilog version requires more effort to do so, especially when the pipeline structure is more complicated than this simple example.

A.2.3 Comparison with Verilog. Following prior work [61] comparing HDLs through a single benchmark, we compare NetX and Verilog on the case-study CPU and additionally include FFT to broaden the comparison. The Verilog versions were reimplemented based on the NetX designs, and both versions use the same Quartus setup [17]. Table 4 reports code size, estimated power, timing, resource usage, and simulation time. Across these two designs, NetX achieves approximately 8–9% higher frequency and uses approximately 5% fewer logic elements, with slightly higher estimated power consumption. Code size varies by design, while simulation takes approximately one quarter of the time required by Icarus Verilog. These results suggest no substantial implementation overhead in the two designs, but do not establish broader performance advantages.

For simulation, we use Icarus Verilog [86] as the baseline because both it and the current NetX simulator interpret the circuit rather than compile it ahead of time into native code. Ahead-of-time (AOT) simulators such as Verilator [75] prioritize speed at the cost of imprecise modeling of certain hardware-specific behaviors (e.g., Verilator needs to approximately synthesize the circuit to perform AOT simulation, which has various known problems with combinational loops, non-blocking assignments, and other features). NetX does not currently support AOT simulation (we leave this as future work), so we choose Icarus Verilog as a baseline with a similar simulation model.

Table 4. Supplementary comparison showing NetX has no substantial overhead over re-implemented Verilog.

	Lang.	LoC	Power (W)		Performance		Area		Sim. Time (s)
			Total	Core Dyn.	Slack (ns)	Freq. (MHz)	LEs	Regs	
FFT	NetX	121	0.203	0.048	11.14	112.97	3998	3627	0.52
	Verilog	241	0.189	0.034	10.33	103.46	4229	3744	2.32
CPU	NetX	1914	0.340	0.148	1.46	58.56	6769	2266	3.02
	Verilog	1874	0.332	0.146	0.76	54.14	7092	2248	12.55

A.2.4 Supplementary User Study. We supplement the preliminary user study in Section 5.3 with details of its methodology and participants' assessments of the language and toolchain.

Methodology. We recruited eight professional hardware developers: five participants from the empirical study (P06, P11, P13, P23, and P24) and three colleagues of previous participants (X01–X03) for additional perspectives. Given the limited availability of professional developers and the effort required to review a new language, the sample size was constrained. Participants were not informed of our hypothesis about consistency and had no conflict of interest.

All participants were *required* to review the tutorial, documentation, and source code of the 12 benchmarks implemented in NetX, although this review required a substantial investment of time and effort—it formed an important part of the evaluation: drawing on their day-to-day hardware-development experience, participants could examine how NetX's constructs were applied across diverse designs and assess the language's design quality. To complement this systematic code-based assessment with hands-on experience, we further encouraged participants to develop projects in NetX. As a result, 6/8 participants implemented 7 projects in NetX as summarized in Table 5, totaling approximately 1,900 lines of NetX code.

After reviewing the materials and completing the hands-on projects, participants then completed a questionnaire covering these four aspects:

(1) Perceived advantages and disadvantages of NetX relative to existing HDLs.
(2) Their overall assessment of the language design, including novelty, effectiveness, and prospects.
(3) Their assessment of the toolchain's functionality and usability.
(4) Their experience with hands-on implementation, where applicable.

The questionnaire provided predefined advantage and disadvantage options alongside open-ended feedback, and participants rated their confidence in each selected property from 1 to 5. Table 6 summarizes these selections, where #P counts participants and ~C is the mean confidence among those selecting the corresponding advantage or disadvantage. The Neutral column counts participants who selected neither assessment for a property; sometimes it indicates an absence of an explicit assessment rather than an explicitly neutral opinion. In particular, reliability was not a predefined option: two participants raised it through open-ended feedback, while the other six did not explicitly assess it, so we report 6 neutral responses for reliability.

Results. Table 6 summarizes participants' assessments. Their detailed comments highlight the following aspects concerning the listed properties:

- *Consistency*. All eight participants selected consistency as an advantage, with a mean confidence of 3.75/5. As reported in Section 5.3, their comments highlight two aspects of this advantage. (1) NetX conveys design intentions directly: "Well-structured NetX code is self-documenting, directly conveying design intentions. It aligns with the intended schematic, which is never the case for other languages like Verilog." (X02); and "Traditional HDLs describe circuits through (behavioral) logic, while NetX describes circuits themselves directly." (P06). (2) NetX makes circuit design more predictable: "NetX can effectively reduce the possibility that a synthesizer misinterprets the design and generates unexpected logic." (P11); and "It is interesting that people can transcribe circuit schematic into NetX code; this correspondence can guide the beginners to better understand how to design a circuit." (P13).
- *Learning*. Six participants selected ease of learning as an advantage (mean confidence: 3.83/5), while one selected it as a disadvantage (1.00/5). Most participants found NetX's programming model intuitive and easy to follow; they quickly grasped the language's core concepts and could read and write NetX code after a short period of study. For example, X03 appreciated circuit-composition operators that avoid large amounts of explicit wiring code while retaining named connections for interfaces where these improve readability. However, P11 emphasized that the training cost might become a barrier to adoption, especially when considering the current university-level and industrial training programs that focus on Verilog and VHDL.
- *Expressiveness*. Four participants selected expressiveness as an advantage (3.50/5) of NetX. For example, P23 found list comprehension impressively useful in many design cases, and appreciated the idea of encapsulating low-level wiring details through zero-cost abstractions; P06 noted that modular design, recursion, and library support offer potential for building larger systems. In

some cases such as the Booth-Wallace multiplier (`wallace8`) where developers especially care about the explicit circuit structure and its performance, they found NetX's schematic model quite expressive, while in other common cases such as describing control signals, there is no obvious advantage or disadvantage of NetX over their familiar HDLs like Verilog.

- *Toolchain maturity.* Participants acknowledged that the prototype provides functionality for compilation, simulation, visualization, and FPGA deployment, while toolchain maturity is still a concern at this stage of language development. Some also raised concerns about simulation performance, reasoning that fine-grained circuit modeling could slow simulation; that said, participants did not conduct experiments comparing simulation performance with other languages, and our preliminary measurements (Section A.2.3) show that NetX's simulation performance is comparable to Verilog in the two designs we tested.
- *Suggestions for future development.* Participants proposed several directions for extending the prototype. P23 and P24 suggested an integrated environment linking source code, circuit schematics, and waveforms: selecting a problematic waveform could locate the corresponding code and schematic, allowing developers to follow circuit connections to investigate its cause. P13 recommended hierarchical visualization so that developers could selectively inspect individual modules, and P23 suggested support for writing verification stimuli and test cases. X02 and X03 suggested more accessible simulation scripting interfaces, including support for languages beyond C++. These suggestions outline opportunities to build on the schematic programming model with more integrated debugging and verification tools.

Table 5. Projects implemented by the participants through hands-on work, showing lines of NetX code.

Project	LoC	Description
`UART`	465	Serial communication protocol
`wallace8`	231	8-bit Booth-Wallace multiplier
`S2F/F2S`	175	Cross-clock-domain (CDC) bridges for the AXI protocol
`afifo`	53	Asynchronous FIFO buffer
`minirv`	201	RISC-style CPU with minimal instructions
`rv32i8`	400	CPU with a reduced RV32I instruction set
`GEMM`	370	Matrix multiplication engine for a specific clustering algorithm

Table 6. Participants' assessments of advantages and disadvantages of NetX. #P denotes the number of participants selecting a property, and ~C denotes their average confidence on a scale from 1 to 5.

Criteria	Positive		Neutral	Negative	
	#P	~C	#P	#P	~C
Consistency	8	3.75	0	0	–
Easy-to-learn	6	3.83	1	1	1.00
Expressiveness	4	3.50	4	0	–
Reliability	2	2.50	6	0	–
Simulation performance	0	–	3	5	3.60
Toolchain maturity	0	–	1	7	4.43